Web Service Architecture: Web services Architecture and its characteristics, core building blocks of web services, standards and technologies available for implementing web services, web services communication, and basic steps of implementing web services.
SOAP: SOAP Model – messages – Encoding – RPC – Alternative SOAP encodings –
Document, RPC, Literal, Encoded – SOAP, Web Services and the REST Architecture
WSDL: Structure – Using SOAP and WSDL. UDDI- UDDI Business Registry –
Specification – Data Structures – Life Cycle Management – Dynamic Access Point
Management.

# What are Web Services?

- The Internet is the worldwide connectivity of hundreds of thousands of computers of various types that belong to multiple networks. On the World Wide Web, a web service is a standardized method for propagating messages between client and server applications. A web service is a software module that is intended to carry out a specific set of functions. Web services in cloud computing can be found and invoked over the network.

  The web service would be able to deliver functionality to the client that invoked the web service.

- A web service is a set of open protocols and standards that allow data to be exchanged between different applications or systems. Web services can be used by software programs written in a variety of programming languages and running on a variety of platforms to exchange data via computer networks such as the Internet in a similar way to inter-process communication on a single computer.
- Any software, application, or cloud technology that uses standardized web protocols (HTTP or HTTPS) to connect, interoperate, and exchange data messages – commonly XML (Extensible Markup Language) – across the internet is considered a web service.
  Web services have the advantage of allowing programs developed in different languages to connect with one another by exchanging data over a web service between clients and servers. A client invokes a web service by submitting an XML request, which the service responds with an XML response.

**Functions of Web Services**

- It's possible to access it via the internet or intranet networks.
- XML messaging protocol that is standardized.
- Operating system or programming language independent.
- Using the XML standard, it is self-describing.
- A simple location approach can be used to locate it.

# Components of Web Services

Web services are software systems designed to enable machine-to-machine interaction over the internet. They are used to share data and functionality between different applications and systems. Here are some of the different types of web services and their importance, which are described below:

- **SOAP (Simple Object Access Protocol):** SOAP is a messaging protocol used for exchanging structured information between applications. It uses XML (Extensible Markup Language) for data exchange and relies on HTTP, SMTP, or other communication protocols. SOAP is important because it allows different platforms and programming languages to communicate with each other.
- **REST (Representational State Transfer):** REST is a style of web services architecture that uses HTTP protocol for data exchange. It uses a uniform interface for client-server communication and can be accessed through simple HTTP requests. REST is important because it provides a simple, scalable, and efficient way to exchange data between systems.
- **XML-RPC:** XML-RPC is a remote procedure call protocol that uses XML to encode its messages. It allows remote systems to communicate and execute functions in a simple and standardized way. XML-RPC is important because it provides a lightweight and easy-to-use mechanism for distributed computing.
- **JSON-RPC**: JSON-RPC is a remote procedure call protocol that uses JSON (JavaScript Object Notation) to encode its messages. It is similar to XML-RPC but uses a more lightweight data format. JSON-RPC is important because it is widely used in modern web applications and provides a simple and efficient way to communicate between systems.
- **Microservices:** Microservices architecture is an approach to building applications as a collection of small, independent services that communicate with each other through APIs. Microservices are important because they allow developers to build applications that are more scalable, flexible, and resilient.

Web Services are an important technology for enabling machine-to-machine communication and data exchange over the internet. Different types of web services have different strengths and weaknesses, and their choice depends on the specific requirements of the system being built.

# Working of Web Services

- Web services are software systems designed to enable communication and data exchange between different applications over the internet. They work by providing a standardized way for applications to share data and functionality, regardless of the platform, programming language, or operating system they are running on.
- When a client application needs to access a web service, it sends a request to the web service using a specific protocol such as SOAP or REST. The request contains information about the data or function the client application wants to access. The web service then processes the request and sends back a response, which contains the requested data or the result of the requested function. Web services typically use XML or JSON to encode data and messages.
- This allows different applications and systems to communicate with each other even if they are using different programming languages or data formats. Web services can be hosted on a server or in the cloud, and can be accessed by any client application with internet connectivity. They are used in a wide variety of applications, including e-commerce, social networking, and financial services, among others.

**Components of Web Services:** Web services are made up of several different components that work together to enable communication and data exchange between different applications over the internet. Here are the main components of web services:

- **XML or JSON:** XML and JSON are the two main data formats used by web services to encode data and messages. XML is a markup language that uses tags to describe the structure of data, while JSON is a lightweight data interchange format that uses key-value pairs.
- **SOAP or REST:** SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) are the two main protocols used by web services to exchange data and messages between applications. SOAP uses XML for data exchange and relies on specific messaging formats, while REST uses HTTP for data exchange and can be accessed through simple HTTP requests.
- **WSDL (Web Services Description Language):** WSDL is an XML-based language used to describe the interface of a web service. It specifies the operations that the service provides, the parameters that can be passed to each operation, and the data types used by the service.
- **UDDI (Universal Description, Discovery, and Integration):** UDDI is a directory service used to publish and discover web services. It provides a standard way for developers to locate and use web services and allows service providers to advertise their services to potential clients.
- **Service provider and service consumer:** A service provider is an entity that creates and publishes the web service, while a service consumer is an entity that accesses the service. The service provider and service consumer communicate with each other using the protocols and data formats specified by the web service.

## Features/Characteristics Of Web Service

Web services have the following features:

**(a) XML Based**: The information representation and record transportation layers of a web service employ XML. There is no need for networking, operating system, or platform binding when using XML. At the middle level, web offering-based applications are highly interoperable.

**(b) Loosely Coupled:** A customer of an internet service provider isn't necessarily directly linked to that service provider. The user interface for a web service provider can change over time without impacting the user's ability to interact with the service provider. A strongly coupled system means that the patron's and server's decisions are inextricably linked, indicating that if one interface changes, the other should be updated as well.

A loosely connected architecture makes software systems more manageable and allows for easier integration between different structures.

**(c) Capability to be Synchronous or Asynchronous:** Synchronicity refers to the client's connection to the function's execution. The client is blocked and the client has to wait for the service to complete its operation, before continuing in synchronous invocations. Asynchronous operations allow a client to invoke a task and then continue with other tasks.

Asynchronous clients get their results later, but synchronous clients get their effect immediately when the service is completed. The ability to enable loosely linked systems requires asynchronous capabilities.

**(d) Coarse-Grained:** Object-oriented systems, such as Java, make their services available through individual methods. At the corporate level, a character technique is far too fine an operation to be useful. Building a Java application from the ground, necessitates the development of several fine-grained strategies, which are then combined into a rough-grained provider that is consumed by either a buyer or a service. Corporations should be coarse-grained, as should the interfaces they expose. Web services generation is an easy approach to define coarse-grained services that have access to enough commercial enterprise logic.

**(e) Supports Remote Procedural Call:** Consumers can use an XML-based protocol to call procedures, functions, and methods on remote objects utilizing web services. A web service must support the input and output framework exposed by remote systems.
Enterprise-wide component development Over the last few years, JavaBeans (EJBs) and.NET Components have become more prevalent in architectural and enterprise deployments. A number of RPC techniques are used to allocate and access both technologies.
A web function can support RPC by offering its own services, similar to those of a traditional role, or by translating incoming invocations into an EJB or.NET component invocation.

**(f) Supports Document Exchanges:** One of XML's most appealing features is its simple approach to communicating with data and complex entities. These records can be as simple as talking to a current address or as complex as talking to an entire book or a Request for Quotation. Web administrations facilitate the simple exchange of archives, which aids incorporate reconciliation.
The web benefit design can be seen in two ways: **(i)** The first step is to examine each web benefit on-screen character in detail. **(ii)** The second is to take a look at the rapidly growing web benefit convention stack.

# Advantages Of Web Service

Using web services has the following advantages:

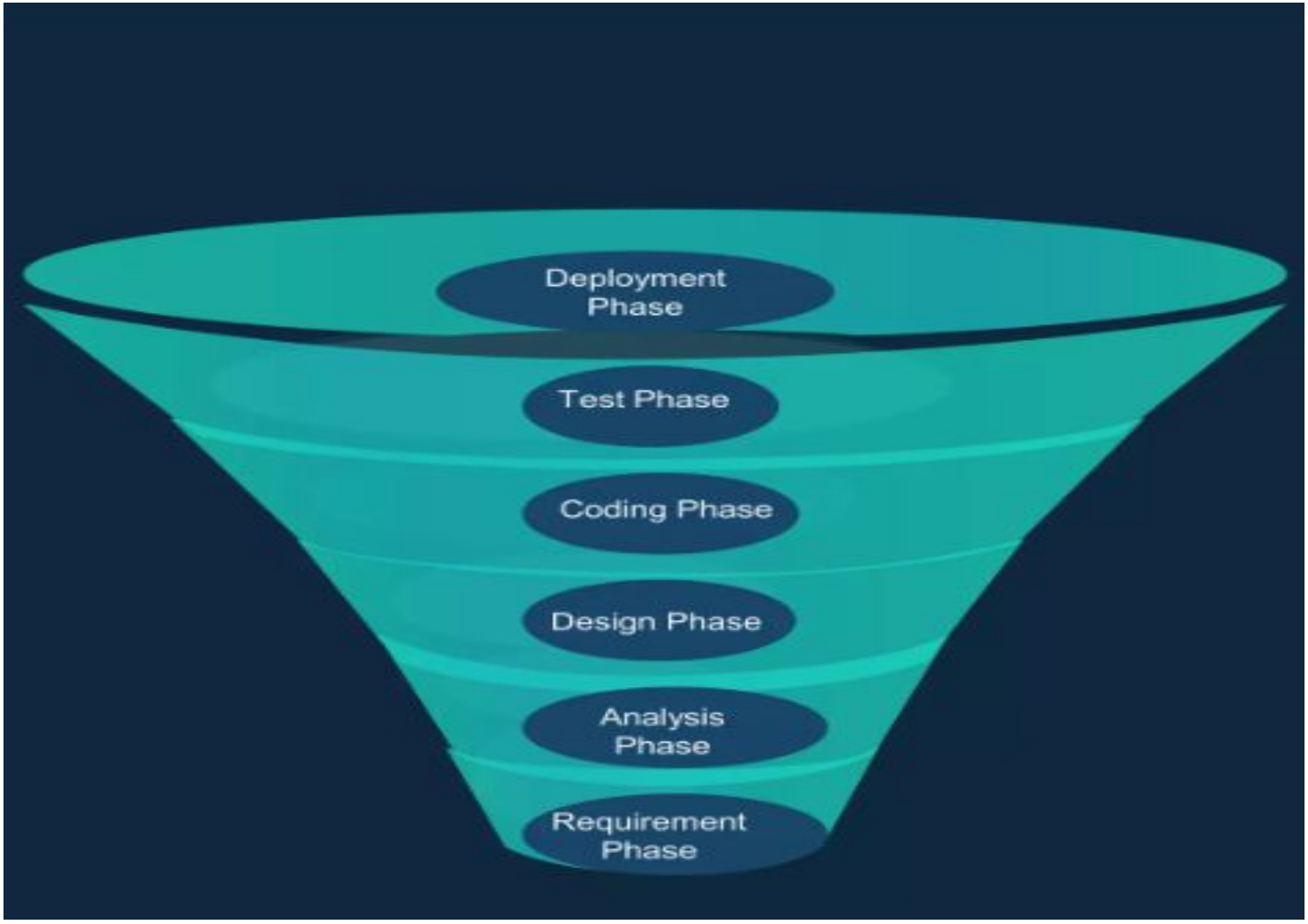
**(a) Business Functions can be exposed over the Internet:** A web service is a controlled code component that delivers functionality to client applications or end-users. This capability can be accessed over the HTTP protocol, which means it can be accessed from anywhere on the internet. Because all apps are now accessible via the internet, Web services have become increasingly valuable. Because all apps are now accessible via the internet, Web services have become increasingly valuable. That is to say, the web service can be located anywhere on the internet and provide the required functionality.

**(b) Interoperability**: Web administrations allow diverse apps to communicate with one another and exchange information and services. Different apps can also make use of web services. A .NET application, for example, can communicate with Java web administrations and vice versa. To make the application stage and innovation self-contained, web administrations are used.

**(c) Communication with Low Cost**: Because web services employ the SOAP over HTTP protocol, you can use your existing low-cost internet connection to implement them. Web services can be developed using additional dependable transport protocols, such as FTP, in addition to SOAP over HTTP.

**(d) A Standard Protocol that Everyone Understands**: Web services communicate via a defined industry protocol. In the web services protocol stack, all four layers (Service Transport, XML Messaging, Service Description, and Service Discovery) use well-defined protocols.

# Web Service Implementation Lifecycle

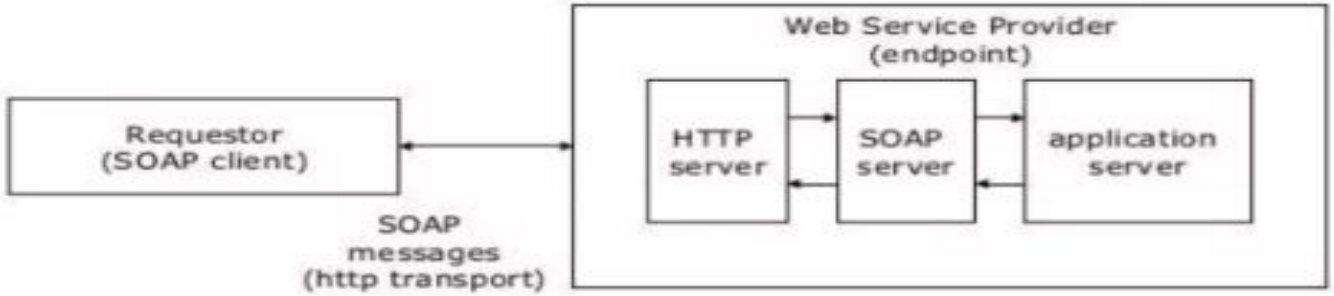

Deployment Phase
Test Phase
Coding Phase
Design Phase
Analysis Phase
Requirement Phase

The Web Service Implementation Lifecycle describes the phases a typical Web Service would undergo, from the identification of the need of the Web Service to the final deployment and usage by the end-users.

The phases identified to be relevant in the Web Service Implementation Lifecycle are: requirements, analysis, design, code, test and deployment.

In each of these phases, Web Service specific activities are carried out. These activities, as well as the roles and responsibilities, and the artifacts will be elaborated in the subsequent sub-sections.

# Web Services Implementation




- Application Server (web service-enabled)
  - provides implementation of services and exposes it through WSDL/SOAP
  - implementation in Java, as EJB, as .NET (C#) etc.
- SOAP server
  - implements the SOAP protocol
- HTTP server
  - standard Web server
- SOAP client
  - implements the SOAP protocol on the client site

- The motivation behind Web Services is to facilitate businesses to interac and integrate with other businesses and clients, without having to go through lengthy integration design and/or to expose its confidential interr application details unnecessarily.

- This is made possible by leveraging on the non-platform dependent and non-programming language dependent XML to describe the data to be exchanged between businesses or between the business and its clients.

- The Web Service Implementation Methodology is iterative and incremental.

- In each iteration, the Web Service would go through all the phases (i.e. requirements, analysis, design, code, testing and finally deployment), thereby developing and refining the Web Services throughout the project lifecycle.

https://www.novell.com/documentation/extend52/Docs/help/Director/books/utoolsUnderstandingServices.html

# SOAP Model

The Simple Object Access Protocol provides a model for distributed processing assuming that a SOAP message is originated at a particular SOAP sender and is ultimately received by an ultimate SOAP receiver, and in the path of the message there could be zero or many SOAP intermediaries.

This processing model defines how any SOAP receiver should process the SOAP message and applies to a single message only isolating itself from any other.

**SOAP Nodes**

A SOAP node is any node that participates in the SOAP message path. It can be the initial SOAP node, in which case it is a SOAP Sender, the ultimate SOAP receiver or a SOAP intermediary. Any SOAP node that receives a SOAP message must perform several processes according to the SOAP specification.

SOAP Roles While processing a SOAP message a SOAP node performs one or several SOAP roles, each of them are identified by a URI(The target namespace of the SOAP service ), also known as the SOAP role name.

SOAP is a way for a program running in one operating system to communicate with a program running in either the same or a different operating system, using HTTP (or any other transport protocol) and XML

# What is SOAP?

SOAP stands for Simple Object Access Protocol

SOAP is a communication protocol

SOAP is for communication between applications

SOAP is a format for sending messages

SOAP communicates via Internet

SOAP is platform independent

SOAP is language independent

SOAP is based on XML
SOAP is simple and extensible
SOAP allows you to get around firewalls
SOAP is a W3C(World Wide Web Consortium) recommendation

**Why SOAP?**

It is important for application development to allow Internet communication between programs.

Today's applications communicate using Remote Procedure Calls (RPC) between objects like DCOM and CORBA, but HTTP was not designed for this. RPC represents a compatibility and security problem; firewalls and proxy servers will normally block this kind of traffic. A better way to communicate between applications is over HTTP, because HTTP is supported by all Internet browsers and servers. SOAP was created to accomplish this.

SOAP provides a way to communicate between applications running on different operating systems, with different technologies and programming languages.

**SOAP Message**

SOAP Building Blocks

A SOAP message is an ordinary XML document containing the following elements:

An Envelope element that identifies the XML document as a SOAP message

A Header element that contains header information

A Body element that contains call and response information

A Fault element containing errors and status information

All the elements above are declared in the default namespace for the SOAP envelope and the default namespace for SOAP encoding and data types is: Syntax Rules

Here are some important syntax rules:

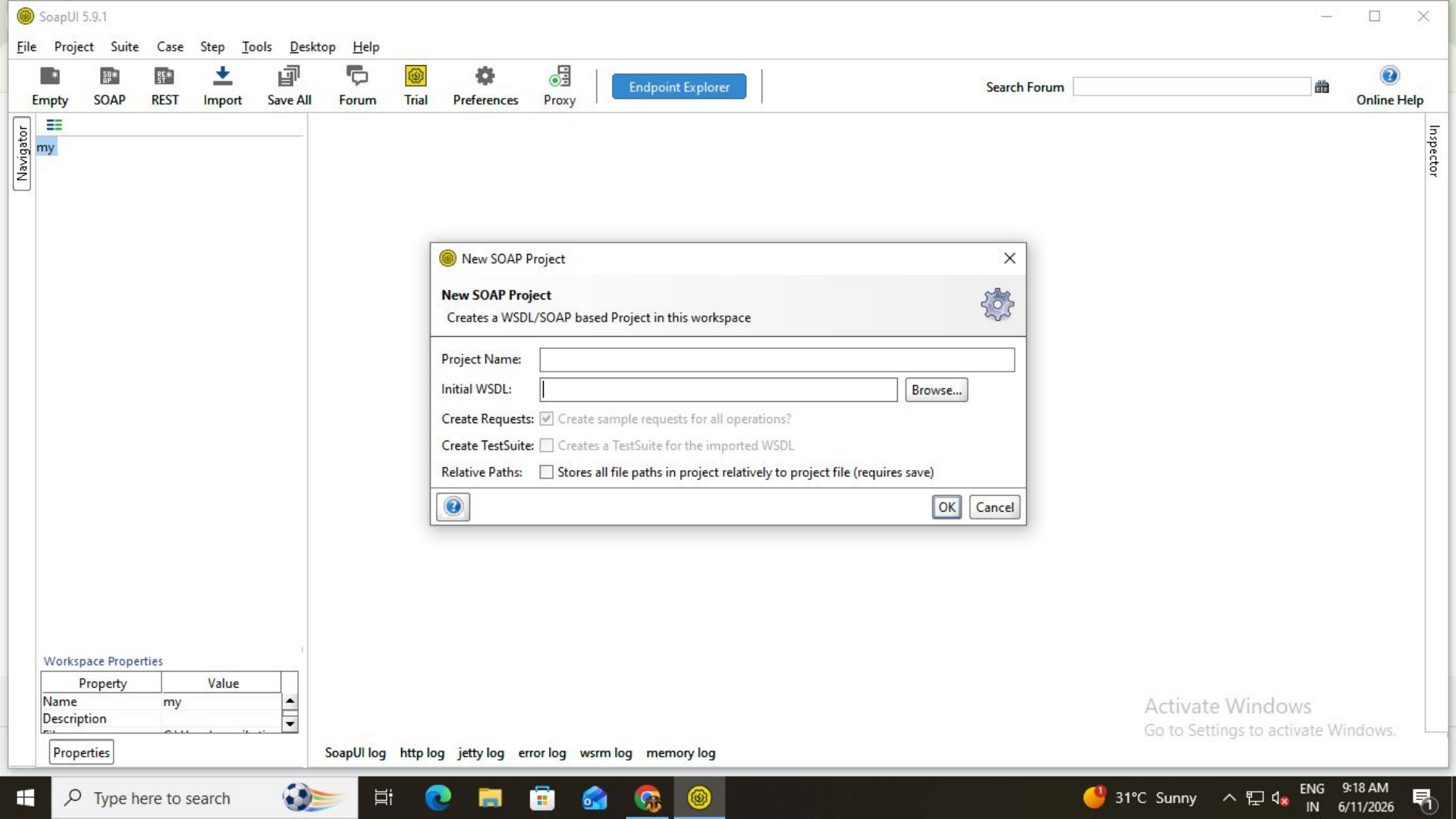
A SOAP message MUST be encoded using XML

A SOAP message MUST use the SOAP Envelope namespace

A SOAP message MUST use the SOAP Encoding namespace

A SOAP message must NOT contain a DTD reference

A SOAP message must NOT contain XML Processing Instructions


SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
Inspector
New SOAP Project
New SOAP Project
Creates a WSDL/SOAP based Project in this workspace
Project Name:
Initial WSDL:
Browse...
Create Requests: Create sample requests for all operations?
Create TestSuite: Creates a TestSuite for the imported WSDL
Relative Paths: Stores all file paths in project relatively to project file (requires save)
OK
Cancel
Workspace Properties
Property
Value
Name
my
Description
Properties
SoapUI log
http log
jetty log
error log
wsrm log
memory log
Activate Windows
Go to Settings to activate Windows.
Type here to search
31°C Sunny
ENG
IN
9:18 AM
6/11/2026

**Skeleton SOAP Message**

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001
/12/soap-envelope"
soap:encodingStyle="http://www.w3.
org/2001/12/soap-encoding">
<soap:Header>
</soap:Header>
<soap:Body>
<soap:Fault>
</soap:Fault>
</soap:Body>
</soap:Envelope>
```

**The SOAP Envelope Element**

The required SOAP Envelope element is the root element of a SOAP message. This element defines the XML document as a SOAP message.

Example

```
<?xml version="1.0"?> <soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soa
p-envelope"
soap:encodingStyle="http://www.w3.org/2001
/12/soap-encoding">
Message information goes here
</soap:Envelope>
```

The xmlns:soap Namespace

Notice the xmlns:soap namespace in the example above. It should always have the value Of: "http://www.w3.org/2001/12/soap-envelope". The namespace defines the Envelope as a SOAP Envelope.

If a different namespace is used, the application generates an error and discards the message.

**The encodingStyle Attribute**

The encodingStyle attribute is used to define the data types used in the document. This attribute may appear on any SOAP element, and applies to the element's contents and all child elements.

A SOAP message has no default encoding.

Syntax

```
soap:encodingStyle="URI"
```

Example

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
Message information goes here
</soap:Envelope>
```

**The SOAP Header Element**

The optional SOAP Header element contains application-specific information (like authentication, payment, etc) about the SOAP message.

If the Header element is present, it must be the first child element of the Envelope element.

Note: All immediate child elements of the Header element must be namespace-qualified.

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Header>
<m:Trans
xmlns:m="http://www.w3schools.com/transaction/"
soap:mustUnderstand="1">234
</m:Trans>
</soap:Header>
```

**REMOTE PROCEDURE CALLS (RPC)**

Remote procedure calls in SOAP are essentially client-server interactions over HTTP where the request and response comply with SOAP encoding rules. The Request-URI (Universal Resource Identifier) in HTTP is typically used at the server end to map to a class or an object, but this is not mandated by SOAP. Additionally, the HTTP header SOAPAction specifies the interface name (a URI) and the name of the method to be called on the server.

The SOAP message is an XML document whose root element, the Envelope, specifies the overall structure of the message, its intended recipient, and other attributes of the message. SOAP specifies a remote procedure call convention, which includes the representation and format to be used for calls and responses. A method call is modeled as a compound data element consisting of a sequence of fields (accessors), one for each parameter. A return structure consists of the return value as well as the out and in/out parameters.

SOAP encoding rules specify the serialization for primitive and application-defined datatypes. Figures 1 and 2 show the request and response structure of a remote procedure call transported as an HTTP request carrying a SOAP payload.

**Figure 1: Example of a SOAP request sent via HTTP.**

```
POST /Temperature HTTP/1.1
Host: www.temperature-service.com
Content-Type: text/xml
Content-Length: 357
SOAPAction: "http://weather.org/query#GetTemperature"
< SOAP-ENV:Envelope
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"

SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
< SOAP-ENV:Body>
< m:GetTemperature xmlns:m="http://weather.org/query">
< longitude>39W< /longitude>
< latitude>62S< /latitude>
< /m:GetTemperature>
< /SOAP-ENV:Body>
< /SOAP-ENV:Envelope>
```

Example of a SOAP response received via HTTP.

```
HTTP/1.1 200 OK
Content-Type: text/xml
Content-Length: 343
< SOAP-ENV:Envelope
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
< SOAP-ENV:Body>
< m:GetTemperatureResponse
xmlns:m="http://weather.org/query">
```

< centigrade>28.4< /centigrade>
< /m:GetTemperatureResponse>
< /SOAP-ENV:Body>
< /SOAP-ENV:Envelope>

SOAP allows hierarchically structured queries and responses, and specifies serialization of primitive string, numeric and date datatypes, and aggregates like arrays and vectors. Sparse arrays, and protocols for sending parts of them are also supported. New types may be defined using the <complexType> construct inside a schema definition.
Overall, SOAP provides many advantages. Unfortunately, its universality comes with a performance penalty: XML messages are textual and so the sizes of its messages are significantly larger than protocols which send binary data. Since a distinguishing characteristic of scientific computation is the need to handle large data sets, the performance of SOAP relative to specialized protocols that can use binary representations is an important issue. The next section tests SOAP performance relative to other communication protocols.

# REST

Web service technology offers several advantages in non-Web environments. For example, Web service technology facilitates the integration of J2EE components with .NET components within an enterprise or department in a straightforward manner.

But as shown, Web services can be implemented in Web environments, too, on top of basic Web technologies such as HTTP, Simple Mail Transfer Protocol (SMTP), and so on.

Representational State Transfer (REST) is a specific architectural style introduced in . Simply put, it is the architecture of the Web.

Consequently, the question arises about how Web services compare to the Web, or how the corresponding underlying architectural styles SOA and REST compare.

## “Representational” in REST

The basic concept of the REST architecture is that of a resource. A resource is any piece of information that you can identify uniquely.

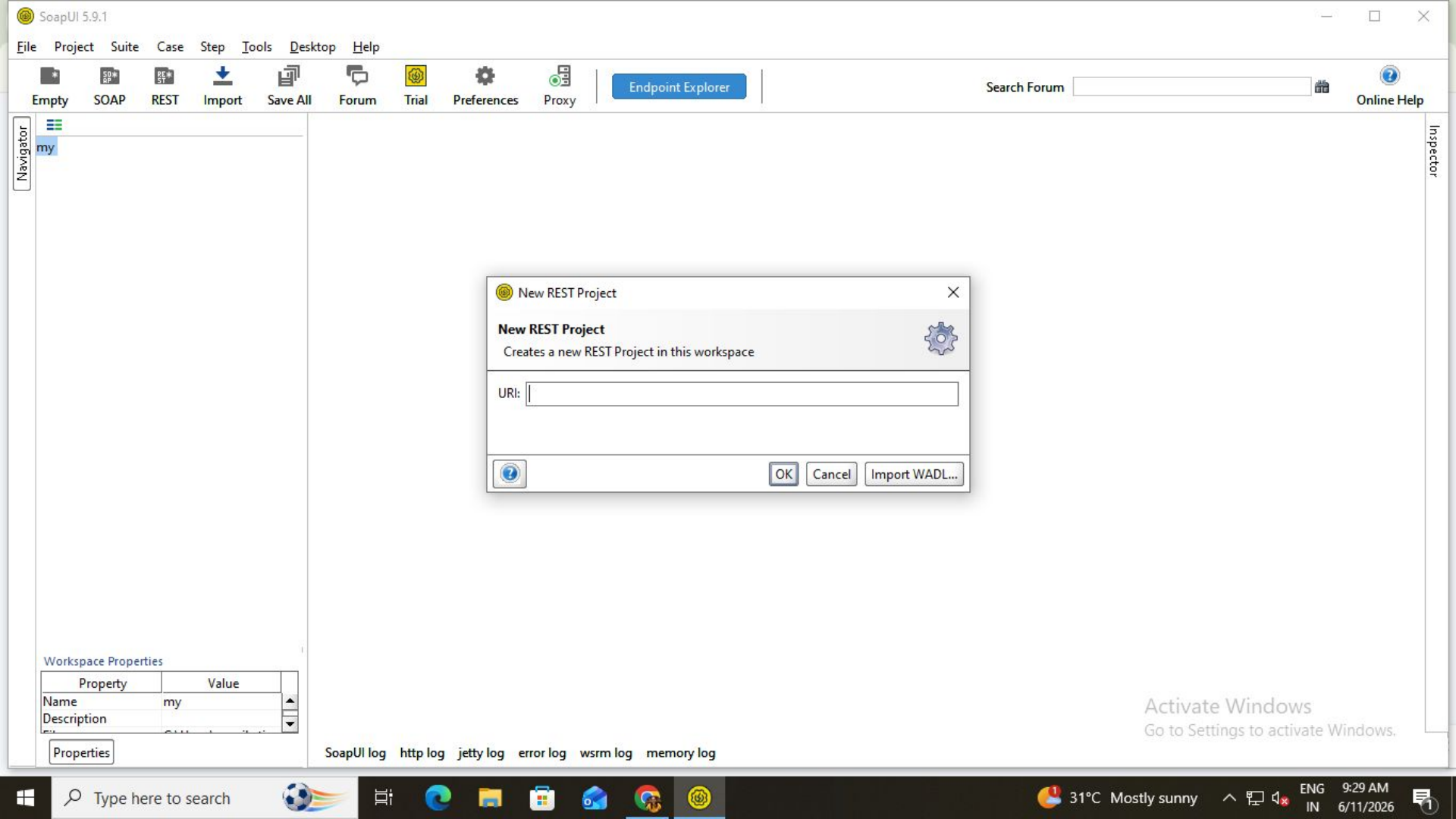
In REST, requesters and services exchange messages that typically contain both data and metadata. The data part of a message corresponds to the resource in a particular representation as described by the accompanying metadata (format), which might also contain additional processing directives. You can exchange a resource in multiple representations. Both communication partners might agree to a particular representation in advance.

For example, the data of a message might be information about the current weather in New York City (the resource). The data might be rendered as an HTML document, and the language in which this document is encoded might be German. This makes up the representation of the resource “current weather in New York City.” The processing directives might indicate that the data should not be cached because it changes frequently.

“State Transfer” in REST

Services in REST do not maintain the state of an interaction with a requester; that is, if an interaction requires a state, all states must be part of the messages exchanged. By being stateless, services increase their reliability by simplifying recovery processing. (A failed service can recover quickly.)

Furthermore, scalability of such services is improved because the services do not need to persist state and do not consume memory, representing active interactions. Both reliability and scalability are required properties of the Web. By following the REST architectural style, you can meet these requirements.


SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
Inspector
New REST Project
New REST Project
Creates a new REST Project in this workspace
URI:
OK Cancel Import WADL...
Workspace Properties
Property Value
Name my
Description
Properties
SoapUI log http log jetty log error log wsrm log memory log
Activate Windows
Go to Settings to activate Windows.
Type here to search
31°C Mostly sunny
ENG IN
9:29 AM 6/11/2026

# REST Interface Structure

REST assumes a simple interface to manipulate resources in a generic manner. This interface basically supports to create, retrieve, update, and delete (CRUD) method.
The metadata of the corresponding messages contains the method name and the identifier of the resource that the method targets. Except for the retrieval method, the message includes a representation of the resource. Therefore, messages are self-describing.
Identifier being included in the messages is fundamental in REST. It implies further benefits of this architectural style.
For example, by making the identifier of the resource explicit, REST furnishes caching strategies at various levels and at proper intermediaries along the message path. An intermediary might determine that it has a valid copy of the target resource available at its side and can satisfy a retrieval request without passing the request further on. This contributes to the scalability of the overall environment.
If you are familiar with HTTP and URIs, you will certainly recognize how REST maps onto these technologies.

## REST and Web Services

At its heart, the discussion of REST versus Web services revolves around the advantage and disadvantages of generic CRUD interfaces and custom-defined interfaces.
Proponents of REST argue against Web service technology because custom-defined Web service interfaces do not automatically result in reliability and scalability of the implementing Web services or cache ability of results, as discussed earlier.
For example, caching is prohibited mainly because neither identifiers of resources nor the semantics of operations are made explicit in messages that represent Web service operations. Consequently, an intermediary cannot determine the target resource of a request message and whether a request represents retrieval or an update of a resource. Thus, an intermediary cannot maintain its cache accordingly.
Proponents of Web service technology argue against REST because quality of service is only rudimental addressed in REST. Scenarios in which SOA is applied require qualities

of services such as reliable transport of messages, transactional interactions, and selective encryption of parts of the data exchanged.

Furthermore, a particular message exchange between a requester and a service might be carried out in SOA over many different transport protocols along its message path—with transport protocols not even supported by the Web. Thus, the tight coupling of the Web architecture to HTTP (and a few other transport protocols) prohibits meeting this kind of end-to-end qualities of service requirement.

Metadata that corresponds to qualities of services cannot—in contrast to what REST assumes—be expected as metadata of the transport protocols along the whole message path. Therefore, this metadata must be part of the payload of the messages. This is exactly what Web service technology addresses from the outset, especially via the header architecture of SOAP.

From an architectural perspective, it is not “either REST or Web services.” Both technologies have areas of applicability. As a rule of thumb, REST is preferable in problem domains that are query intense or that require exchange of large grain chunks of data. SOA in general and Web service technology as described in this book in particular is preferable in areas that require asynchrony and various qualities of services. Finally, SOA- based custom interfaces enable straightforward creation of new services based on choreography.

You can even mix both architectural styles in a pure Web environment. For example, you can use a regular HTTP GET request to solicit a SOAP representation of a resource that the URL identifies and specifies in the HTTP message. In that manner, benefits from both approaches are combined. The combination allows use of the SOAP header architecture in the response message to built-in quality of service that HTTP does not support (such as partial encryption of the response). It also supports the benefits of REST, such as caching the SOAP response. Representational State Transfer (REST) is an architectural style that abstracts the architectural elements within a distributed hypermedia system. REST ignores the details of component implementation and protocol syntax in order to focus on the roles of components, the constraints upon their interaction with other components, and their interpretation of significant data elements. REST has emerged as a predominant web API design model.

## Scope of Applicability of Web Service

As indicated throughout the first three chapters of this book, Web service technology provides a uniform usage model for components/services, especially within the context of heterogeneous distributed environments. Web service technology also virtualizes resources (that is, components that are software artifacts or hardware artifacts). Both are achieved by shielding idiosyncrasies of the different environments that host those components. This shielding can occur by dynamically selecting and binding those components and by hiding the communication details to properly access those components.
Furthermore, interactions between a requester and a service might show configurable qualities of service, such as reliable message transport, transaction protection, message-level security, and so on.

These qualities of services are not just ensured between two participants but between any number of participants in heterogeneous environments.

Given this focus, the question about the scope of applicability of SOA in general and Web service technology in particular is justified. As with most architectural questions, there is no crisp answer, no hard or fast rule to apply. Given this, common sense should prevail. For example:

SOA is not cost effective for organizations that have small application portfolios or those whose new interface requirements are not enough to benefit from SOA.

SOA does not benefit organizations that have relatively static application portfolios that are already fully interfaced.

If integration of components within heterogeneous environments or dynamically changing component configurations is at the core of the problem being addressed, consider SOA and Web service technology. SOA offers potentially significant benefits to organizations with large application portfolios that undergo frequent change (lots of mergers and acquisitions or frequent switching of service providers).

If reusability of a function (in the sense of making it available to all kinds of requesters) is important, providing the function as a Web service is a good approach.

Currently, the XML footprint and parsing cost at both ends of a message exchange does take up time and resources. If high performance is the most important criterion for primary implementation, consider the use of Web service technology with care. Use of binary XML for interchange might help this, but currently there are no agreed-upon standards for this.

Similarly, if the problem in hand is within a homogeneous environment, and interoperation with other external environments is not an issue, Web service technology might not have significant benefit.

# WSDL

What is WSDL?

WSDL stands for Web Services Description Language

WSDL is written in XML

WSDL is an XML document

WSDL is used to describe Web services

WSDL is also used to locate Web services

WSDL is a W3C recommendation

WSDL is a document written in XML. The document describes a Web service. It specifies the location of the service and the operations (or methods) the service exposes.

The WSDL Document Structure

A WSDL document describes a web service using these major elements:

Element Defines

<types> The data types used by the web service

<message> The messages used by the web service

<portType> The operations performed by the web service

<binding> The communication protocols used by the web service

The main structure of a WSDL document looks like this:

```
<definitions>
<types>
definition of types.....
</types>
<message>
definition of a message....
</message>
<portType>
definition of a port.....
</portType>
<binding>
definition of a binding....
</binding>
</definitions>
```

A **WSDL** document can also contain other elements, like extension elements, and a service element that makes it possible to group together the definitions of several web services in one single WSDL document.

**WSDL Ports**

The <portType> element is the most important WSDL element.

It describes a web service, the operations that can be performed, and the messages that are involved.

The <portType> element can be compared to a function library (or a module, or a class) in a traditional programming language.

**WSDL Messages**

The <message> element defines the data elements of an operation.

Each message can consist of one or more parts. The parts can be compared to the parameters of a function call in a traditional programming language.

**WSDL Types**

The <types> element defines the data types that are used by the web service.

For maximum platform neutrality, WSDL uses XML Schema syntax to define data types.

**WSDL Bindings**

The <binding> element defines the message format and protocol details for each port.

WSDL Example

This is a simplified fraction of a WSDL document:

```
<message name="getTermRequest">
<part name="term" type="xs:string"/>
</message>
<message name="getTermResponse">
<part name="value" type="xs:string"/>
</message>
<portType name="glossaryTerms">
<operation name="getTerm">
<input message="getTermRequest"/>
<output message="getTermResponse"/>
</operation>
</portType>
```

In this example the <portType> element defines "glossaryTerms" as the name of a port, and "getTerm" as the name of an operation.

The "getTerm" operation has an input message called "getTermRequest" and an output message called "getTermResponse".

The <message> elements define the parts of each message and the associated data types.

Compared to traditional programming, glossaryTerms is a function library, "getTerm" is a function with "getTermRequest" as the input parameter, and getTermResponse as the return parameter.

**WSDL Ports**

The <portType> element is the most important WSDL element. It defines a web service, the operations that can be performed, and the messages that are involved.

The port defines the connection point to a web service. It can be compared to a function library (or a module, or a class) in a traditional programming language. Each operation can be compared to a function in a traditional programming language.

Operation Types

The request-response type is the most common operation type, but WSDL defines four types:

Type Definition

One-way The operation can receive a message but will not return a response

Request-response The operation can receive a request and will return a response Solicit-response The operation can send a request and will wait for a response Notification The operation can send a message but will not wait for a response

One-Way Operation

A one-way operation example:

```
<message name="newTermValues">
<part name="term" type="xs:string"/>
<part name="value" type="xs:string"/>
</message>
<portType name="glossaryTerms">
<operation name="setTerm">
<input name="newTerm" message="newTermValues"/>
</operation>
</portType >
```

In the example above, the port "glossaryTerms" defines a one-way operation called "setTerm".

The "setTerm" operation allows input of new glossary terms messages using a "newTermValues" message with the input parameters "term" and "value". However, no output is defined for the operation.

## Request-Response Operation

A request-response operation example:
<message name="getTermRequest">
<part name="term" type="xs:string"/>
</message>
<message name="getTermResponse">
<part name="value" type="xs:string"/>
</message>
<portType name="glossaryTerms">
<operation name="getTerm">
<input message="getTermRequest"/>
<output message="getTermResponse"/>
</operation>
</portType>
In the example above, the port "glossaryTerms" defines a request-response operation called "getTerm". The "getTerm" operation requires an input message called "getTermRequest" with a parameter called "term", and will return an output message called "getTermResponse" with a parameter called "value".

## Binding to SOAP

A request-response operation example:

```
<message name="getTermRequest">
<part name="term" type="xs:string"/>
</message>
<message name="getTermResponse">
<part name="value" type="xs:string"/>
</message>
<portType name="glossaryTerms">
<operation name="getTerm">
<input message="getTermRequest"/>
<output message="getTermResponse"/>
</operation>
</portType>
<binding type="glossaryTerms" name="b1">
<soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http" />
<operation>
<soap:operation soapAction="http://example.com/getTerm"/>
<input><soap:body use="literal"/></input>
<output><soap:body use="literal"/></output>
</operation>
</binding>
```

The binding element has two attributes - name and type.
The name attribute (you can use any name you want) defines the name of the binding, and the type attribute points to the port for the binding, in this case the "glossaryTerms" port.
The soap:binding element has two attributes - style and transport.
The style attribute can be "rpc" or "document". In this case we use document.
The transport attribute defines the SOAP protocol to use. In this case we use HTTP.
The operation element defines each operation that the port exposes.
For each operation the corresponding SOAP action has to be defined. You must also specify how the input and output are encoded. In this case we use "literal".

# How do SOAP APIs and REST APIs work?

SOAP is an older technology that requires a strict communication contract between systems. New web service standards have been added over time to accommodate technology changes, but they create additional overheads. REST was developed after SOAP and inherently solves many of its shortcomings. REST web services are also called *RESTful web services*.

## SOAP APIs

SOAP is a protocol that defines rigid communication rules. It has several associated standards that control every aspect of the data exchange. For example, here are some standards SOAP uses:

- Web Services Security (WS-Security) specifies security measures like using unique identifiers called *tokens*
- Web Services Addressing (WS-Addressing) requires including routing information as metadata
- WS-ReliableMessaging standardizes error handling in SOAP messaging
- Web Services Description Language (WSDL) describes the scope and function of SOAP web services

When you send a request to a SOAP API, you must wrap your HTTP request in a *SOAP envelope*. This is a data structure that modifies the underlying HTTP content with SOAP request requirements. Due to the envelope, you can also send requests to SOAP web services with other transport protocols, like TCP or Internet Control Message Protocol (ICMP). However, SOAP APIs and SOAP web services always return XML documents in their responses.

**REST APIs**

REST is a software architectural style that imposes six conditions on how an API should work. These are the six principles REST APIs follow:

1. *Client-server architecture*. The sender and receiver are independent of each other regarding technology, platforming, programming language, and so on.
2. *Layered*. The server can have several intermediaries that work together to complete client requests, but they are invisible to the client.
3. *Uniform interface*. The API returns data in a standard format that is complete and fully useable.
4. *Stateless*. The API completes every new request independently of previous requests.
5. *Cacheable*. All API responses are cacheable.
6. *Code on demand*. The API response can include a code snippet if required.

You send REST requests using HTTP verbs like *GET* and *POST*. Rest API responses are typically in JSON but can also be of a different data format.

# What are the similarities between SOAP and REST?

To build applications, you can use many different programming languages, architectures, and platforms. It's challenging to share data between such varied technologies because they have different data formats. Both SOAP and REST emerged in an attempt to solve this problem.

You can use SOAP and REST to build APIs or communication points between diverse applications. The terms *web service* and *API* are used interchangeably. However, APIs are the broader category. Web services are a special type of API.

Here are other similarities between SOAP and REST:

- They both describe rules and standards on how applications make, process, and respond to data requests from other applications
- They both use HTTP, the standardized internet protocol, to exchange information
- They both support SSL/TLS for secure, encrypted communication

You can use either SOAP or REST to build secure, scalable, and fault-tolerant distributed systems.

| | SOAP | REST |
|---|---|---|
| tands for | Simple Object Access Protocol | Representational State Transfer |
| Vhat is it? | SOAP is a protocol for communication between applications | REST is an architecture style for designing communication interfaces. |
| esign | SOAP API exposes the operation. | REST API exposes the data. |
| ransport Protocol | SOAP is independent and can work with any transport protocol. | REST works only with HTTPS. |
| ata format | SOAP supports only XML data exchange. | REST supports XML, JSON, plain text, HTML. |
| erformance | SOAP messages are larger, which makes communication slower. | REST has faster performance due to smaller messages and caching support. |
| calability | SOAP is difficult to scale. The server maintains state by storing all previous messages exchanged with a client. | REST is easy to scale. It's stateless, so every message is processed independently of previous messages. |
| ecurity | SOAP supports encryption with additional overheads. | REST supports encryption without affecting performance. |

# UDDI

**What is UDDI** (Universal Description, Discovery, and Integration)

 UDDI is a platform-independent framework for describing services, discovering businesses, and integrating business services by using the Internet.

- UDDI stands for Universal Description, Discovery and Integration
- UDDI is a directory for storing information about web services
- UDDI is a directory of web service interfaces described by WSDL
- UDDI communicates via SOAP
- UDDI is built into the Microsoft .NET platform

## What is UDDI Based On?

 UDDI uses World Wide Web Consortium (W3C) and Internet Engineering Task Force (IETF) Internet standards such as XML, HTTP, and DNS protocols.
 UDDI uses WSDL to describe interfaces to web services
 Additionally, cross platform programming features are addressed by adopting SOAP, known as XML Protocol messaging specifications found at the W3C Web site.

## UDDI Benefits

 Any industry or businesses of all sizes can benefit from UDDI.
 Before UDDI, there was no Internet standard for businesses to reach their customers and partners with information about their products and services. Nor was there a method of how to integrate into each other's systems and processes.
 Problems the UDDI specification can help to solve:

Making it possible to discover the right business from the millions currently online • Defining how to enable commerce once the preferred business is discovered • Reaching new customers and increasing access to current customers

• Expanding offerings and extending market reach

• Solving customer-driven need to remove barriers to allow for rapid participation in the global Internet economy

• Describing services and business processes programmatically in a single, open, and secure environment

## How can UDDI be Used

 If the industry published an UDDI standard for flight rate checking and reservation, airlines could register their services into an UDDI directory. Travel agencies could then search the UDDI directory to find the airline's reservation interface. When the interface is found, the travel agency can communicate with the service immediately because it uses a well-defined reservation interface.

## UDDI BUSINESS REGISTRY

## Relationship of UDDI Core Data Types

## Business Service

 Each business Service structure represents a logical grouping of Web services. At the service level, there is still no technical information provided about those services; rather, this structure allows the ability to assemble a set of services under a common rubric. Each businessService is the logical child of a single businessEntity. Each businessService contains descriptiveinformation – again, names, descriptions and classification
 information -- outlining the purpose of the individual Web services found within it. For example, a businessService structure could contain a set of Purchase Order Web services (submission, confirmation and notification) that are provided by a business.

## SPECIFICATION

- The UDDI project also defines a set of XML Schema definitions that describe the data formats used by the various specification APIs. These documents are all available for download at www.uddi.org. The current version of all specification groups is Version 2.0.
- The specifications include the following − UDDI Replication,

- UDDI Operators,
- UDDI Programmer's API, and
- UDDI Data Structures

**UDDI Replication**

 This document describes the data replication processes and interfaces to which a registry operator must conform to achieve data replication between sites. This specification is not a programmer's API; it defines the replication mechanism used among UBR nodes.

**UDDI Operators**

 This document outlines the behavior and operational parameters required by the UDDI node operators. This specification defines data management requirements to which operators must adhere.

**UDDI Programmer's API**

 This specification defines a set of functions that all UDDI registries support for inquiring about services hosted in a registry and for publishing information about a business or a service to a registry. This specification defines a series of SOAP messages containing XML documents that a UDDI registry accepts, parses, and responds to. This specification, along with the UDDI XML API schema and the UDDI Data Structure specification, makes up a complete programming interface to a UDDI registry.

## UDDI Data Structures

 This specification covers the specifics of the XML structures contained within the SOAP messages defined by the UDDI Programmer's API. This specification defines five core data structures and their relationships with one another.

 The UDDI XML API schema is not contained in a specification; rather, it is stored as an XML Schema document that defines the structure and datatypes of the UDDI data structures.

 UDDI includes an XML Schema that describes the following data structures − •

businessEntity

• businessService
• bindingTemplate
• tModel
• publisherAssertion

**businessEntity Data Structure**

 The business entity structure represents the provider of web services. Within the UDDI registry, this structure contains information about the company itself, including contact information, industry categories, business identifiers, and a list of services provided.

 Here is an example of a fictitious business's UDDI registry entry −

```
<businessEntity businessKey =
"uuid:C0E6D5A8-C446-4f01-99DA
70E212685A40"
      operator = "http://www.ibm.com"
        authorizedName = "John Doe">
 <name>Acme Company</name>
 <description>
 We create cool Web services
 </description>
 <contacts>
 <contact useType = "general info">
 <description>General Information</description>
 <personName>John Doe</personName>
 <phone>(123) 123-1234</phone>
 <email>jdoe@acme.com</email>
 </contact>
 </contacts>
 <businessServices>
 </businessServices>
 <identifierBag>
 <keyedReference tModelKey =
"UUID:8609C81E-EE1F-4D5A-B20
2- 3EB13AD01823"
 name = "D-U-N-S" value = "123456789" />
 </identifierBag>
 <categoryBag>
```

```
<keyedReference tModelKey =
                "UUID:C0B9FE13-179F-413D-8A5B
                5004DB8E5BB2"
name = "NAICS" value = "111336" />
</categoryBag>
</businessEntity>
```

**businessService Data Structure**

 The business service structure represents an individual web service provided by the business entity. Its description includes information on how to bind to the web service, what type of web service it is, and what taxonomical categories it belongs to.

 Here is an example of a business service structure for the Hello World web service.

```
<businessService serviceKey = "uuid:D6F1B765-BDB3-4837-828D-8284301E5A2A" businessKey =
"uuid:C0E6D5A8-C446-4f01-99DA-70E212685A40"> <name>Hello World Web Service</name>
 <description>A friendly Web service</description>
 <bindingTemplates>
 </bindingTemplates>
 <categoryBag />
</businessService>
```

Notice the use of the Universally Unique Identifiers (UUIDs) in the businessKey and serviceKey attributes. Every business entity and business service is uniquely identified in all the UDDI registries through the UUID assigned by the registry when the information is first entered.

**bindingTemplate Data Structure**

 Binding templates are the technical descriptions of the web services represented by the business service structure. A single business service may have multiple binding templates. The binding template represents the actual implementation of the web service.

 Here is an example of a binding template for Hello World.

```
<bindingTemplate serviceKey = "uuid:D6F1B765-BDB3-4837-828D-8284301E5A2A" bindingKey =
"uuid:C0E6D5A8-C446-4f01-99DA-70E212685A40">
```

<description>Hello World SOAP Binding</description>
<accessPoint URLType =
"http">http://localhost:8080</accessPoint>
<tModelInstanceDetails>
<tModelInstanceInfo tModelKey =
"uuid:EB1B645F-CF2F-491f-811A 4868705F5904">
<instanceDetails>
<overviewDoc>
<description>
references the description of the WSDL service definition
</description>

<overviewURL>
http://localhost/helloworld.wsdl

</overviewURL>
</overviewDoc>
</instanceDetails>
</tModelInstanceInfo>
</tModelInstanceDetails>
</bindingTemplate>

As a business service may have multiple binding templates, the service may specify different implementations of the same service, each bound to a different set of protocols or a different network address.

## tModel Data Structure

 tModel is the last core data type, but potentially the most difficult to grasp. tModel stands for technical model.

 tModel is a way of describing the various business, service, and template structures stored within the UDDI registry. Any abstract concept can be registered within the UDDI as a tModel. For instance, if you define a new WSDL port type, you can define a tModel that represents that port type within the UDDI. Then, you can specify that a given business service implements that port type by associating the tModel with one of that business service's binding templates. Here is an example of a tModel representing the Hello World Interface port type.

<tModel tModelKey = "uuid:xyz987..." operator =
"http://www.ibm.com" authorizedName = "John Doe">
<name>HelloWorldInterface Port Type</name>
<description>
An interface for a friendly Web service
</description>

<overviewDoc>
<overviewURL>
http://localhost/helloworld.wsdl
</overviewURL>
</overviewDoc>
</tModel>

## publisherAssertion Data Structure

 This is a relationship structure putting into association two or more businessEntity structures according to a specific type of relationship, such as subsidiary or department.

 The publisherAssertion structure consists of the three elements: fromKey (the first businessKey), toKey (the second businessKey), and keyedReference.  The keyedReference designates the asserted relationship type in terms of a keyName keyValue pair within a tModel, uniquely referenced by a tModelKey.

```
<element name = "publisherAssertion" type =
"uddi:publisherAssertion" /> <complexType
name = "publisherAssertion">
 <sequence>
 <element ref = "uddi:fromKey" />
 <element ref = "uddi:toKey" />
 <element ref = "uddi:keyedReference" />
 </sequence>
                                    </complexType>
```

SOAP: SOAP Model – messages – Encoding – RPC – Alternative SOAP encodings – Document, RPC, Literal, Encoded – SOAP, Web Services and the REST Architecture

# Basics of SOAP - Simple Object Access Protocol

Simple Object Access Protocol(SOAP) is a network protocol for exchanging structured data between nodes. It uses XML format to transfer messages. It works on top of application layer protocols like HTTP and SMTP for notations and transmission. SOAP allows processes to communicate throughout platforms, languages, and operating system, since protocols like HTTP are already installed on all platforms. SOAP was designed by Bob Atkinson, Don Box, Dave Winer, and Mohsen Al-Ghosein at Microsoft in 1998. SOAP was maintained by the XML Protocol Working Group of the World Wide Web Consortium until 2009.

## Message Format

SOAP message transmits some basic information as given below

- Information about message structure and instructions on processing it.
- Encoding instructions for application-defined data types.
- Information about Remote Procedure Calls and their responses.

The message in XML format contains four parts-

- **Envelope:** This specifies that the XML message is a SOAP message. A SOAP message is an XML document containing a header and a body, both encapsulated within the envelope. Any fault is included within the body of the message.
- **Header:** This part is optional. When present, it can provide crucial information about the applications.
- **Body:** This contains the actual message being transmitted. Faults are contained within the body tags.
- **Fault:** This optional section contains the status of the application and any errors. It should not appear more than once in a SOAP message.

# Sample Message

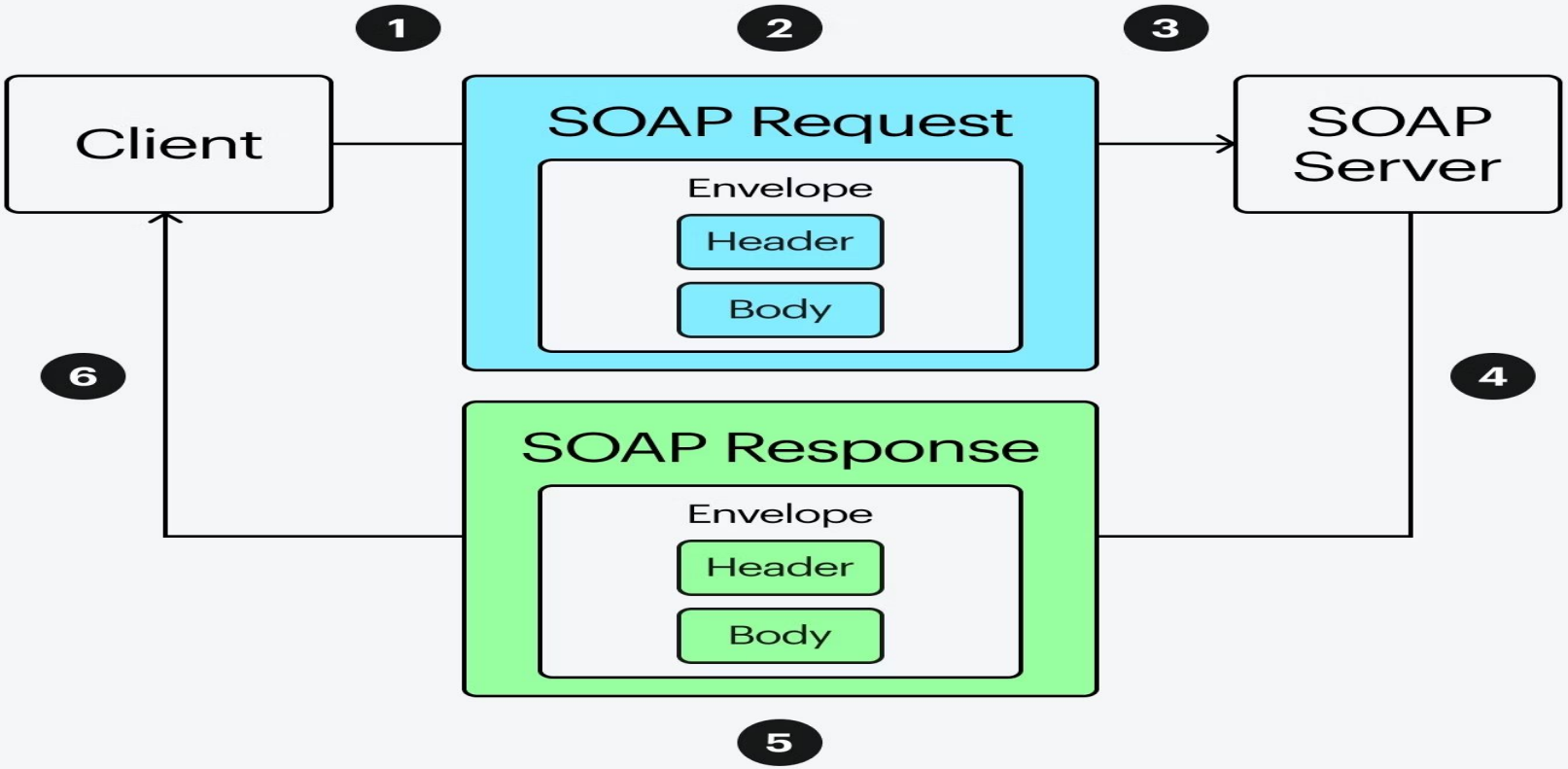
Content-Type: application/soap+xml

```
<env:Envelope
xmlns:env="https://www.w3.org/2003/05/soap-envel
ope">

   <env:Header>

      <m:GetLastTradePrice xmlns:m="Some-URI"
/>

   </env:Header>

   <env:Body>

      <symbol xmlns:p="Some-URI"
>DIS</symbol>

   </env:Body>

</env:Envelope>
```

## Advantages of SOAP

- SOAP is a light weight data interchange protocol because it is based on XML.
- SOAP was designed to be OS and Platform independent.
- It is built on top of HTTP which is installed in most systems.
- It is suggested by W3 consortium which is like a governing body for the Web.
- SOAP is mainly used for Web Services and Application Programming Interfaces (APIs).

# SOAP Exchange Mechanism



# 1. SOAP Message Structure Example

A SOAP message contains:

- Envelope
- Header (optional)
- Body
- Fault (optional)

## Example Code:

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
<soap:Header>
<auth>

<username>admin</username>

<password>1234</password>
</auth>
</soap:Header>
<soap:Body>
<getStudent>
<id>101</id>
</getStudent>
</soap:Body>
</soap:Envelope>
```

# 2. SOAP RPC Style Example

RPC style treats SOAP like a remote method call.

### SOAP Request

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <addNumbers>
            <a>10</a>
            <b>20</b>
        </addNumbers>
    </soap:Body>
</soap:Envelope>
```

## SOAP Response

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <addNumbersResponse>
            <result>30</result>
        </addNumbersResponse>
    </soap:Body>
</soap:Envelope>
```

## 3. SOAP Encoding Example (Encoded Style)

SOAP encoding defines how data types are serialized.

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
    <soap:Body>
        <student>
            <name xsi:type="xsd:string">Monika</name>
            <age xsi:type="xsd:int">22</age>
        </student>
    </soap:Body>
</soap:Envelope>
```

## 4. Document Style SOAP Example

Document style sends XML documents instead of method calls.

### Request

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <StudentRequest>
            <StudentID>101</StudentID>
        </StudentRequest>
    </soap:Body>
</soap:Envelope>
```

## Response

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <StudentResponse>
            <Name>Monika</Name>
            <Course>MCA</Course>
        </StudentResponse>
    </soap:Body>
</soap:Envelope>
```

# 5. Literal SOAP Example

Literal means XML follows the exact XSD schema.

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <BookRequest xmlns="http://example.com/book">
            <BookID>5001</BookID>
        </BookRequest>
    </soap:Body>
</soap:Envelope>
```

## 6. RPC/Encoded Example

This combines RPC style with SOAP encoding.

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
    <soap:Body>
        <multiply>
            <x xsi:type="xsd:int">5</x>
            <y xsi:type="xsd:int">4</y>
        </multiply>
    </soap:Body>
</soap:Envelope>
```

## 7. Document/Literal Example (Most Common in Web Services)

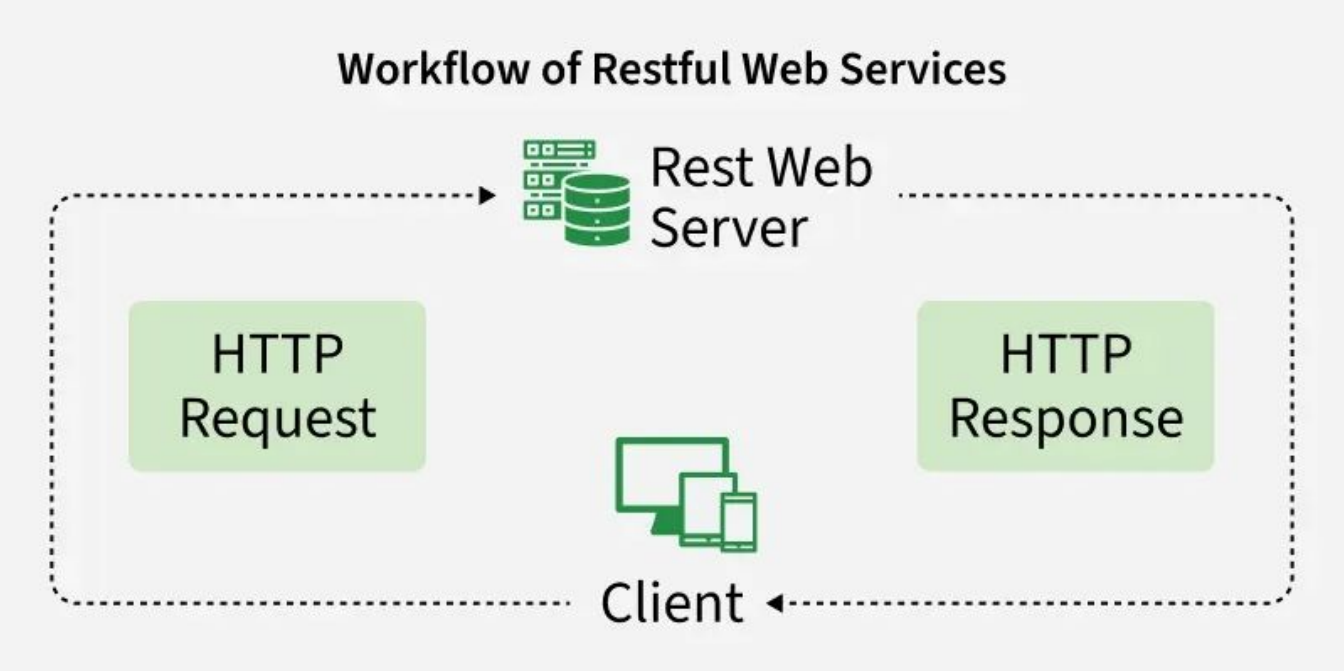
```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <OrderRequest xmlns="http://example.com/order">
            <OrderID>9001</OrderID>
            <Customer>Monika</Customer>
        </OrderRequest>
    </soap:Body>
</soap:Envelope>
```

# Short Difference Table

| Type | Description |
| --- | --- |
| RPC | Works like calling a method/function |
| Document | Sends XML documents |
| Encoded | Uses SOAP encoding rules |
| Literal | Follows exact XML schema (XSD) |
| RPC/Encoded | Old SOAP style |
| Document/Literal | Modern and most widely used |

# RESTful Web Services

RESTful Web Services are a way of designing and developing web services that use REST (Representational State Transfer) principles. They enable applications to communicate over the web using standard HTTP methods, such as GET, POST, PUT and DELETE. REST is lightweight, stateless and widely used in modern web and mobile applications.



# What is REST

REST or Representational State Transfer is an architectural style that can be applied to web services to create and enhance properties like performance, scalability and modifiability.

REST became popular due to its simplicity and adaptability

- **Cross-Platform Communication:** Applications built in different languages and environments can easily talk to each other.
- **Mobile-Friendly:** REST works smoothly on mobile devices without extra effort.
- **Cloud Compatibility:** Most modern cloud platforms and architectures are based on REST principles.

RESTful Architecture

- **Resources via HTTP:** Every resource is accessed using standard HTTP methods (GET, POST, PUT, DELETE).
- **Stateless:** Each request is independent; the server doesn't store client session data.
- **Client/Server:** The client (browser/app) sends requests and the server responds with data.
- **Layered:** Requests may pass through intermediaries like load balancers or security layers.
- **Caching:** Responses can be cached for faster performance.

Principles of RESTful Applications

**1. URI Resource Identification**

- Each resource (user, product, file) should have a unique URI (e.g., /users/1).
- URIs help with easy discovery and access.

**2. Uniform Interface**

- REST uses a fixed set of operations: GET, POST, PUT, DELETE.
- This makes REST simple and consistent.

**3. Self-Descriptive Messages**

- Data can be sent/received in formats like JSON, XML, HTML, PDF or plain text.
- Metadata helps with caching, error handling, authentication, etc.

**4. Hyperlinks for State Interactions (HATEOAS)**

- RESTful responses can include links that guide clients on what they can do next.
- Example: A response for /users/1 may also include a link to /users/1/orders.

## RESTful Web Service Example

Steps:

Create Dynamic Web Project

Configure Apache Tomcat

Convert Project to Maven Project > Configure → Convert to Maven Project

Add Maven Dependencies > pom.xml

# POM.XML

```
<dependencies>
    <dependency>
        <groupId>org.glassfish.jersey.containers</groupId>
        <artifactId>jersey-container-servlet</artifactId>
        <version>2.35</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.inject</groupId>
        <artifactId>jersey-hk2</artifactId>
        <version>2.35</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-json-jackson</artifactId>
        <version>2.35</version>
    </dependency>
</dependencies>
```

Create Package > myrest
Create Student.java

```
package myrest;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String
course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

# Create StudentService.java

```
package myrest;

import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.PathParam;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("/student")
public class StudentService {

    @GET
    @Path("/{id}")

@Produces(MediaType.APPLICATIO
N_JSON)
 public Student
getStudent(@PathParam("i
d") int id) {

        Student s = new
Student();
        s.setId(id);
        s.setName("Monika");
        s.setCourse("MCA");

        return s;
    }
}
```

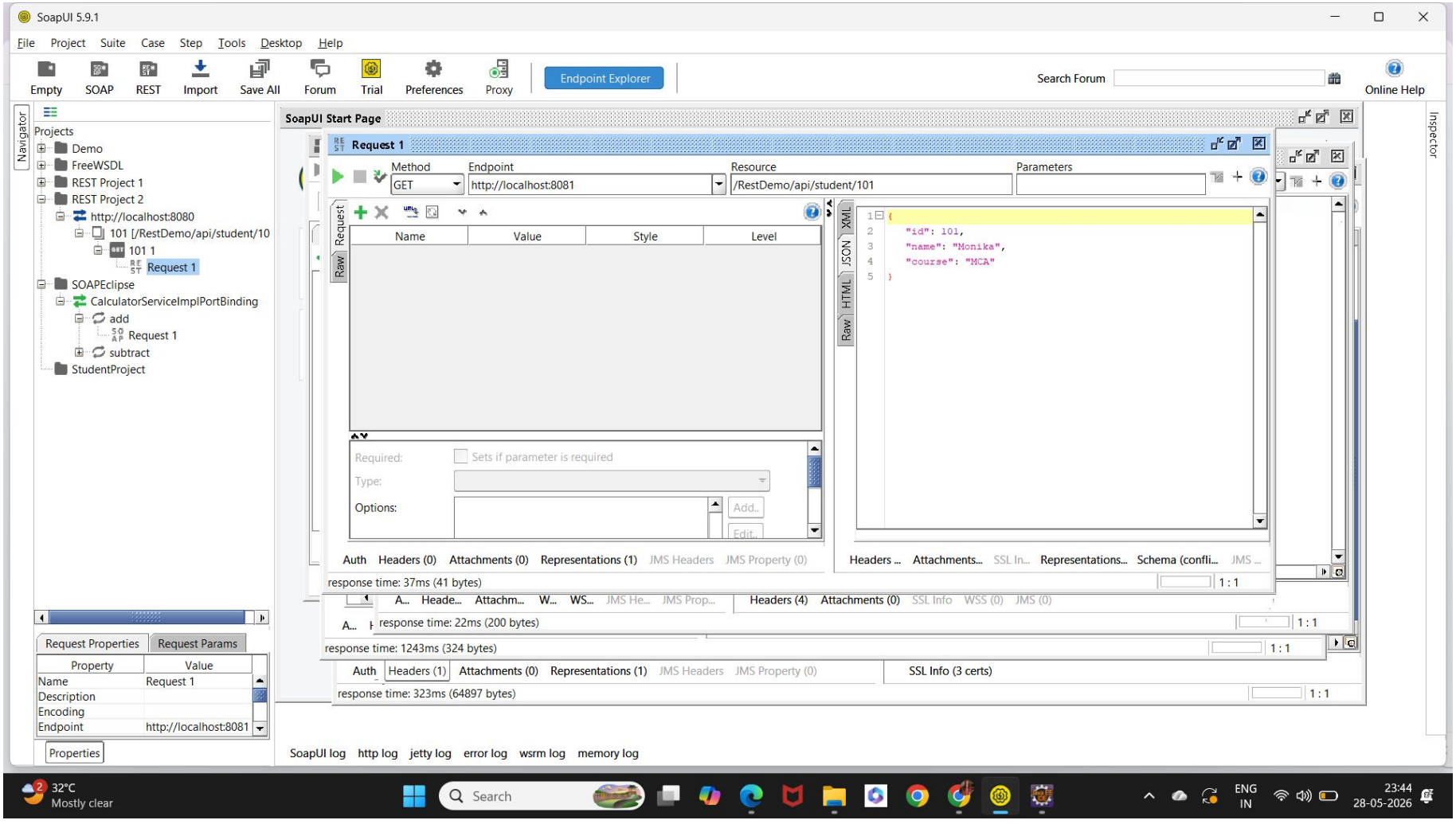
# Configure web.xml

```
<web-app>

    <servlet>
        <servlet-name>Jersey REST
Service</servlet-name>

        <servlet-class>

org.glassfish.jersey.servlet.ServletContainer
        </servlet-class>

        <init-param>
            <param-name>

jersey.config.server.provider.packages
            </param-name>
    <param-value>myrest</param-value>
        </init-param>


<load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>Jersey REST
Service</servlet-name>
        <url-pattern>/api/*</url-pattern>
    </servlet-mapping>

</web-app>
```

- Run Project
- Run As → Run on Server
- Test in Browser > http://localhost:8080/RestDemo/api/student/101


- Create REST Project in SOAP UI
- File → New REST Project > Enter URL > http://localhost:8080/RestDemo/api/student/101 > Click OK
- Send Request > Click green ▸ button.
- Response will be


```
{
  "id":101,
  "name":"Monika",
  "course":"MCA"
}
```

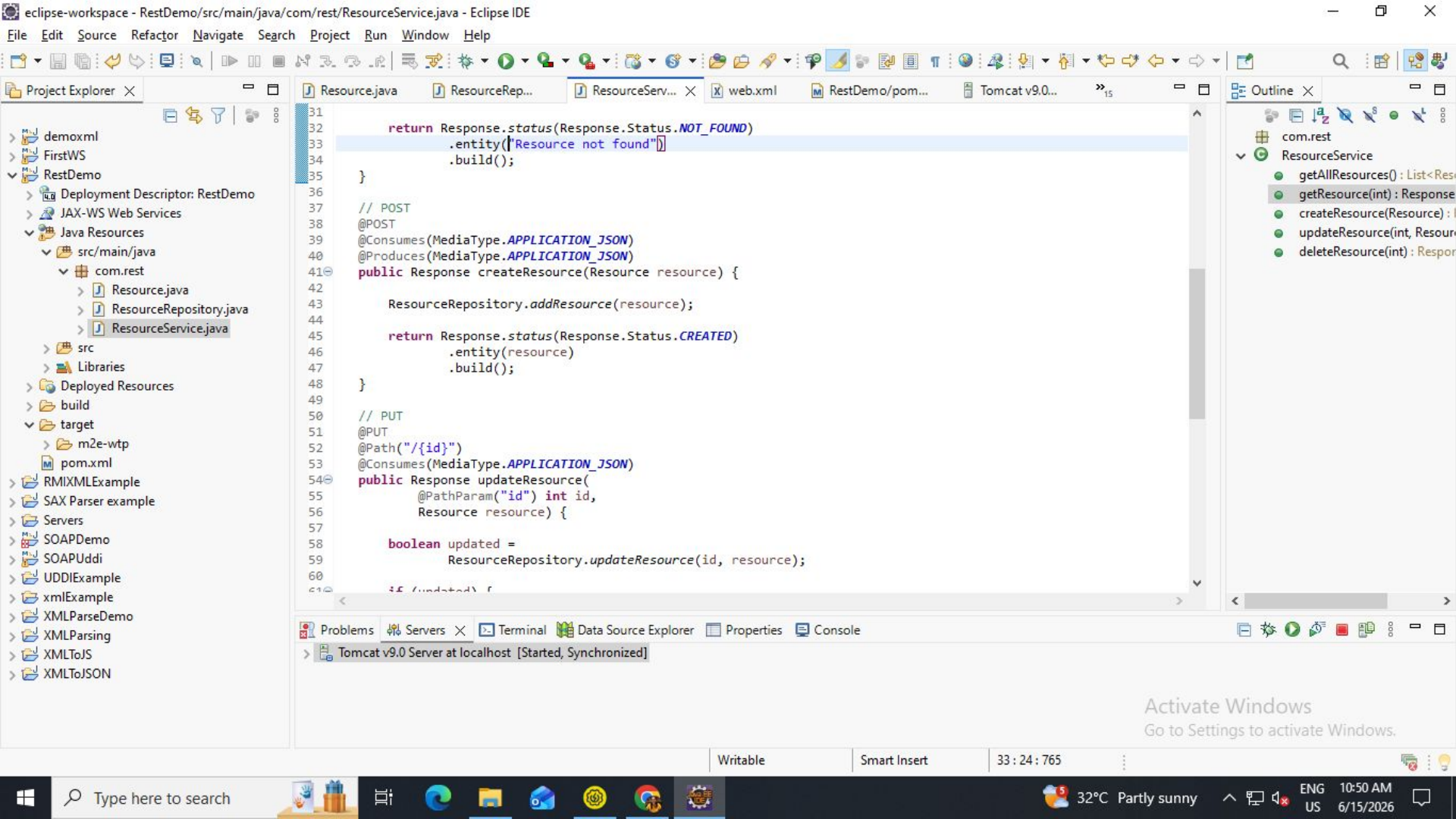

SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
Projects
Demo
FreeWSDL
REST Project 1
REST Project 2
http://localhost:8080
101 [/RestDemo/api/student/10
101 1
Request 1
SOAPEclipse
CalculatorServiceImplPortBinding
add
Request 1
subtract
StudentProject
SoapUI Start Page
Request 1
Method
Endpoint
Resource
Parameters
GET
http://localhost:8081
/RestDemo/api/student/101
Request
Raw
Name
Value
Style
Level
XML
JSON
HTML
Raw
{
"id": 101,
"name": "Monika",
"course": "MCA"
}
Required:
Sets if parameter is required
Type:
Options:
Add..
Edit..
Auth Headers (0) Attachments (0) Representations (1) JMS Headers JMS Property (0)
Headers ... Attachments... SSL In... Representations... Schema (confli... JMS ...
response time: 37ms (41 bytes)
1 : 1
Headers (4) Attachments (0) SSL Info WSS (0) JMS (0)
response time: 22ms (200 bytes)
1 : 1
response time: 1243ms (324 bytes)
1 : 1
Auth Headers (1) Attachments (0) Representations (1) JMS Headers JMS Property (0)
SSL Info (3 certs)
response time: 323ms (64897 bytes)
1 : 1
Request Properties
Request Params
Property
Value
Name
Request 1
Description
Encoding
Endpoint
http://localhost:8081
Properties
Inspector
SoapUI log http log jetty log error log wsrm log memory log
32°C
Mostly clear
Search
ENG
IN
23:44
28-05-2026

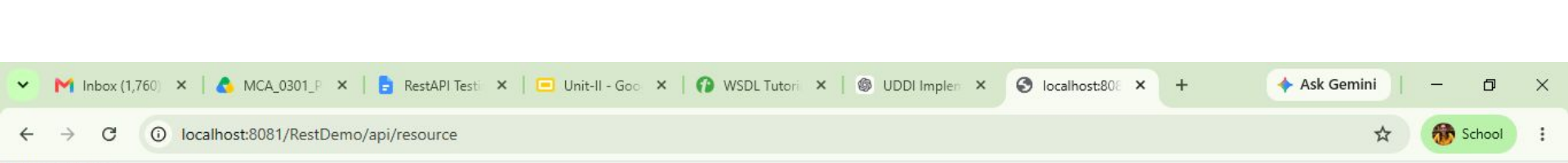
# CRUD Operation using SOAP and Eclipse IDE with Restful Service

If we want to use and Test REST API with our application then we have to implement through Dynamic Web Project in Eclipse.

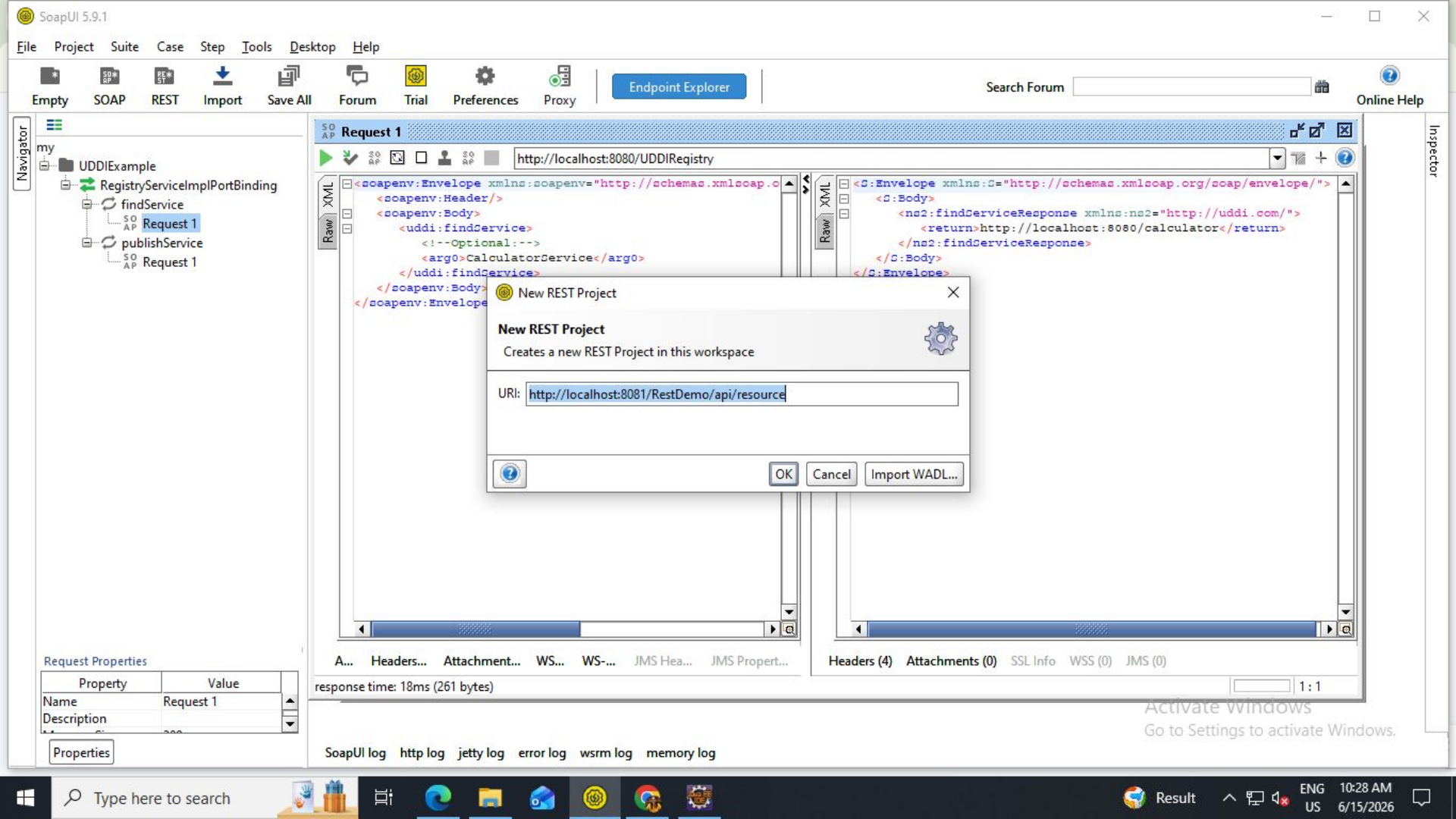
Because we are testing methods like : GET, POST, PUT, Delete etc.

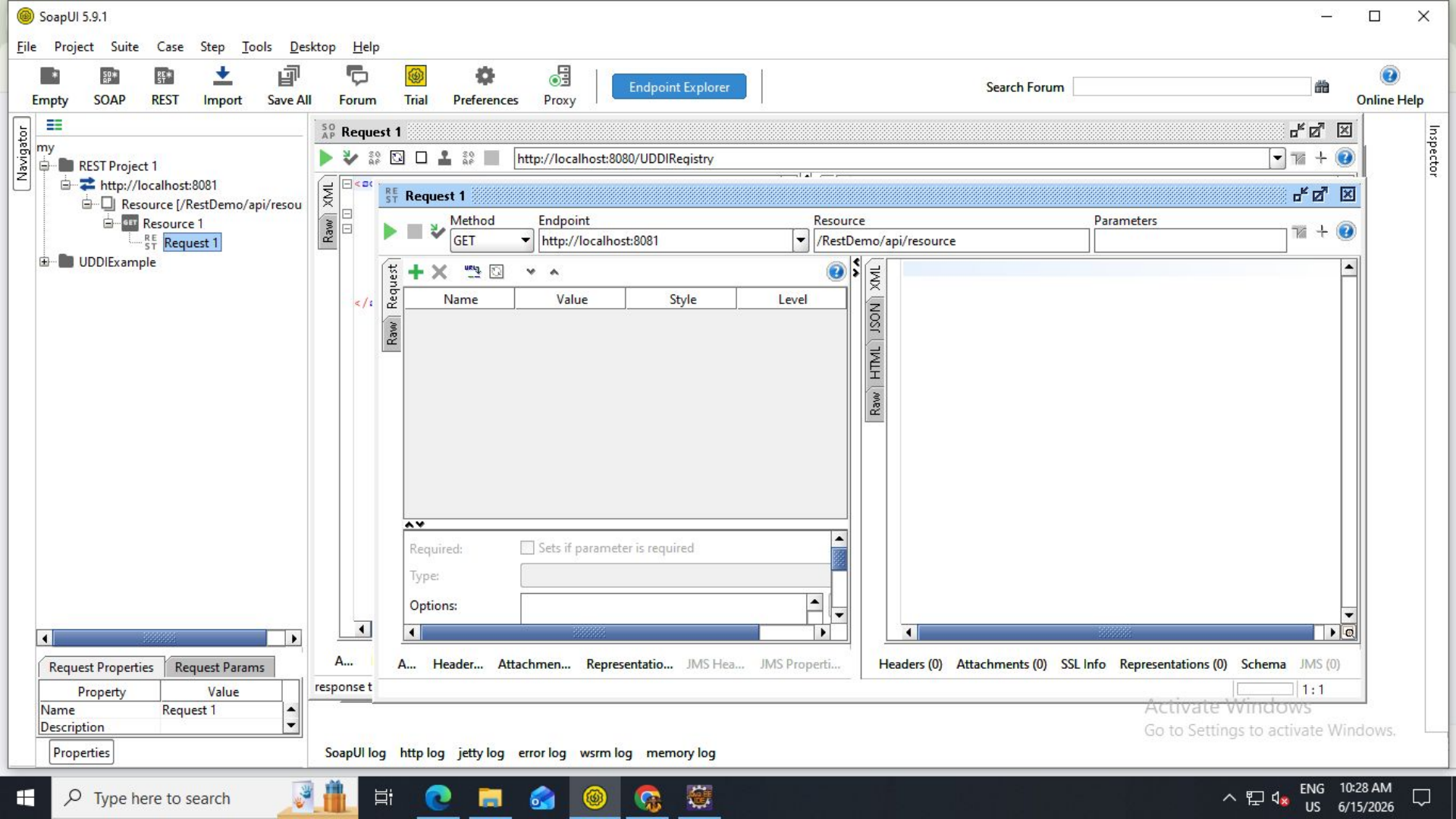
1. **Create Dynamic Web Project** > Write all the given code of files with Web.xml and pom.xml file. For Web.xml file right check on generate deployment descriptor file while creating project.
2. Convert to maven project
3. Maven > Update project
4. Start Tomcat server > It will download all the maven libraries also.
5. Paste the http://localhost:8081/RestDemo/api/resource on browser
6. Now Create REST Project and give the same URL to create project link
7. Now Test All methods in REST UI
8. Post method will require data to post with new ID
9. Put method will require data to modify with ID
10. Delete method will require the deletion data ID.

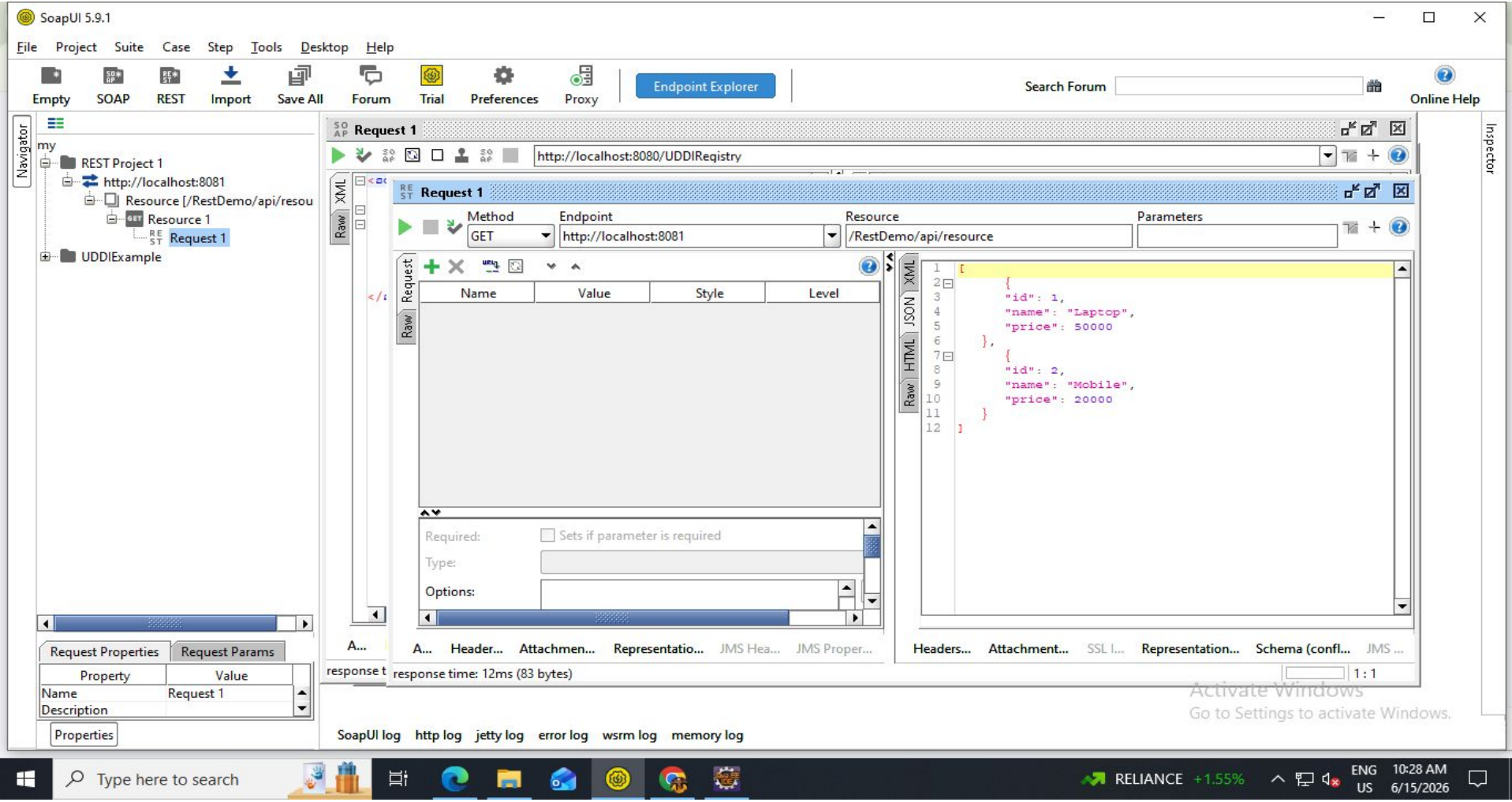

eclipse-workspace - RestDemo/src/main/java/com/rest/ResourceService.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
demoxml
FirstWS
RestDemo
Deployment Descriptor: RestDemo
JAX-WS Web Services
Java Resources
src/main/java
com.rest
Resource.java
ResourceRepository.java
ResourceService.java
src
Libraries
Deployed Resources
build
target
m2e-wtp
pom.xml
RMIXMLExample
SAX Parser example
Servers
SOAPDemo
SOAPUddi
UDDIExample
xmlExample
XMLParseDemo
XMLParsing
XMLToJS
XMLToJSON
Resource.java
ResourceRep...
ResourceServ...
web.xml
RestDemo/pom...
Tomcat v9.0...
return Response.status(Response.Status.NOT_FOUND)
.entity("Resource not found")
.build();
}
// POST
@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response createResource(Resource resource) {
ResourceRepository.addResource(resource);
return Response.status(Response.Status.CREATED)
.entity(resource)
.build();
}
// PUT
@PUT
@Path("/{id}")
@Consumes(MediaType.APPLICATION_JSON)
public Response updateResource(
@PathParam("id") int id,
Resource resource) {
boolean updated =
ResourceRepository.updateResource(id, resource);
Outline
com.rest
ResourceService
getAllResources() : List<Res
getResource(int) : Response
createResource(Resource) :
updateResource(int, Resour
deleteResource(int) : Respor
Problems
Servers
Terminal
Data Source Explorer
Properties
Console
Tomcat v9.0 Server at localhost [Started, Synchronized]
Activate Windows
Go to Settings to activate Windows.
Writable
Smart Insert
33 : 24 : 765
Type here to search
32°C Partly sunny
ENG
US
10:50 AM
6/15/2026



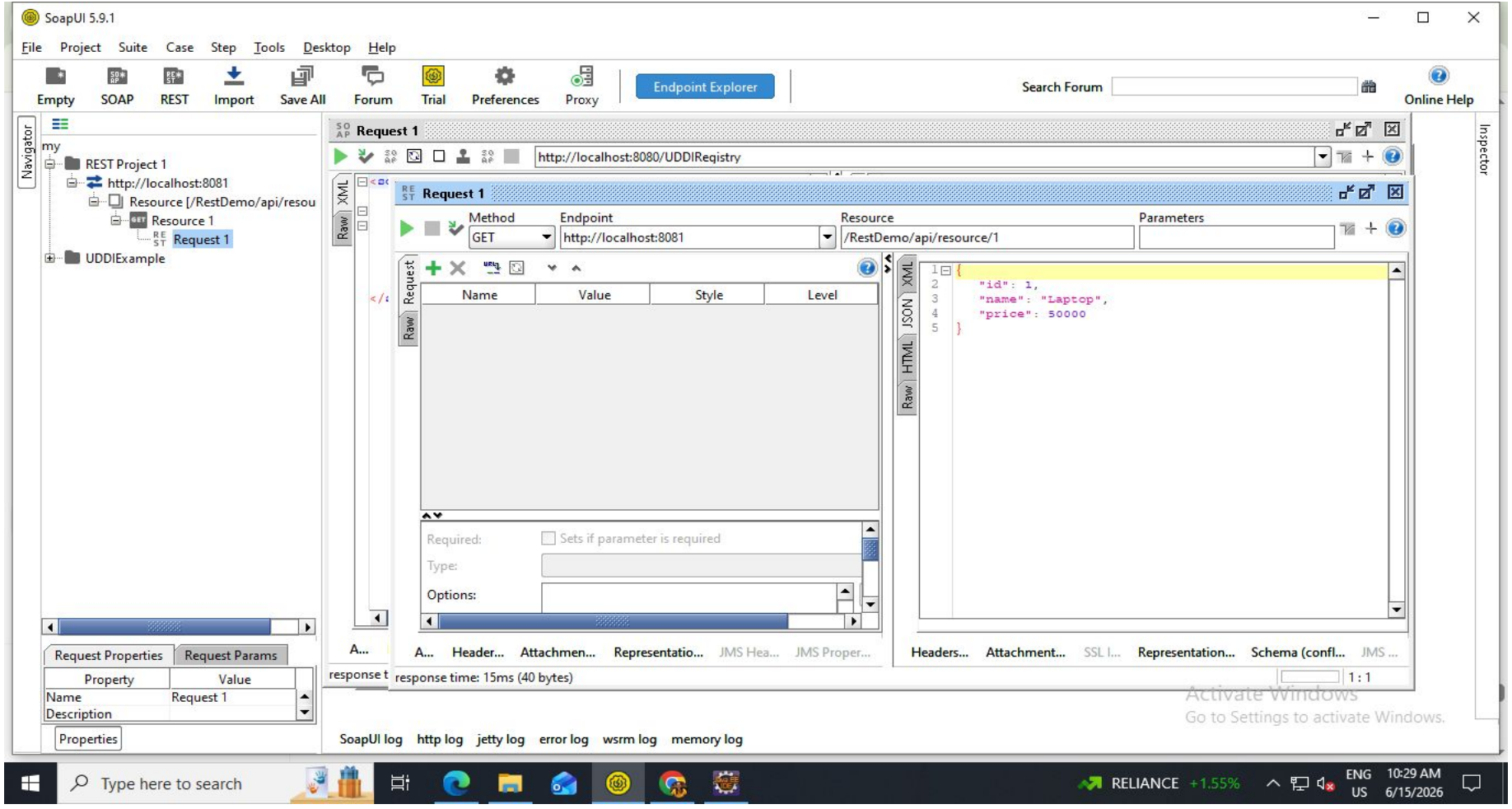

[{"id":1,"name":"Laptop","price":50000.0},{"id":2,"name":"Mobile","price":20000.0}]

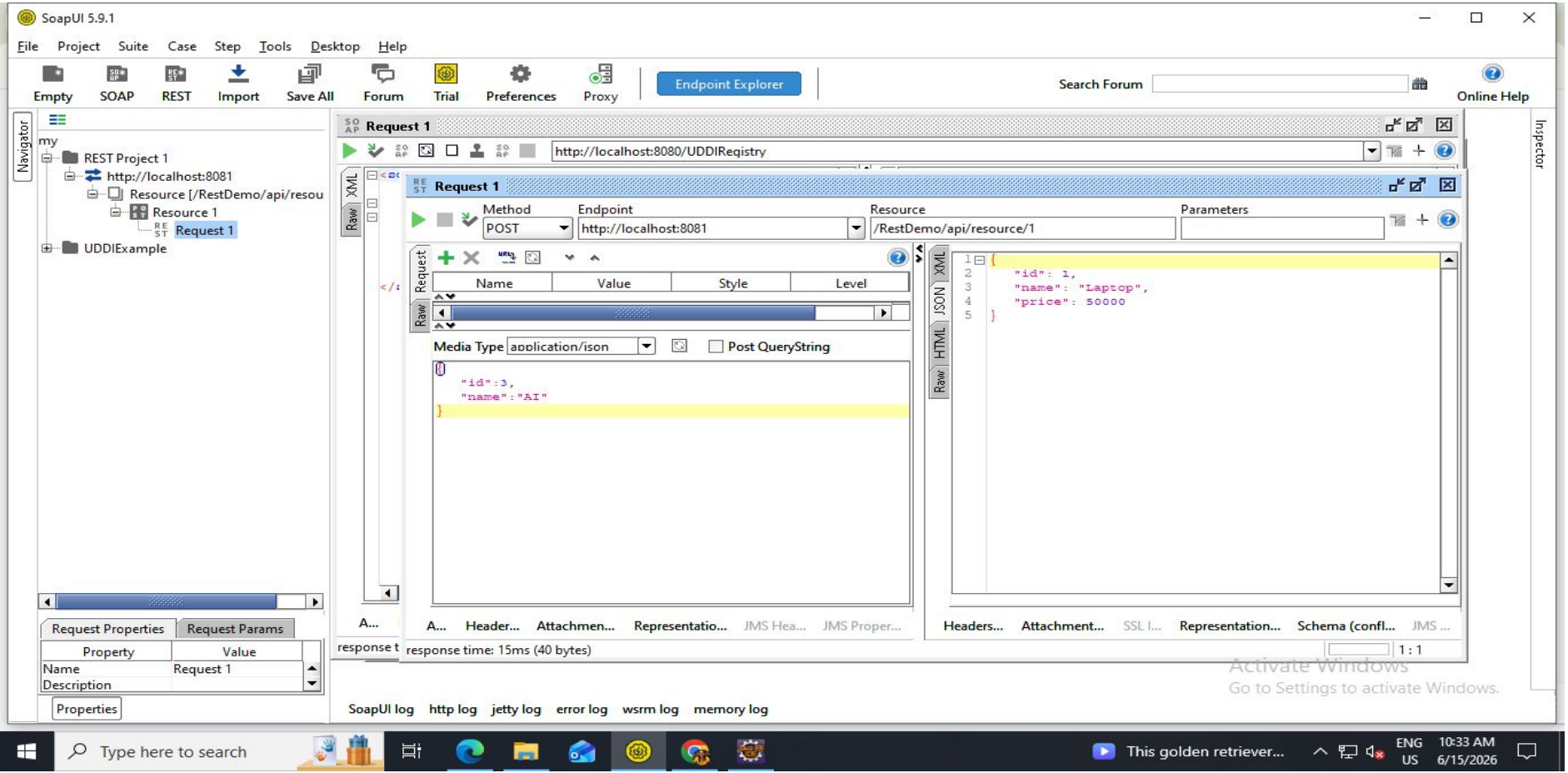

SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
UDDIExample
RegistryServiceImplPortBinding
findService
Request 1
publishService
Request 1
Request 1
http://localhost:8080/UDDIRegistry
XML
Raw
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.o
<soapenv:Header/>
<soapenv:Body>
<uddi:findService>
<!--Optional:-->
<arg0>CalculatorService</arg0>
</uddi:findService>
</soapenv:Body>
</soapenv:Envelope
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
<ns2:findServiceResponse xmlns:ns2="http://uddi.com/">
<return>http://localhost:8080/calculator</return>
</ns2:findServiceResponse>
</S:Body>
</S:Envelope>
New REST Project
New REST Project
Creates a new REST Project in this workspace
URI: http://localhost:8081/RestDemo/api/resource
OK
Cancel
Import WADL...
A... Headers... Attachment... WS... WS-... JMS Hea... JMS Propert...
Headers (4) Attachments (0) SSL Info WSS (0) JMS (0)
response time: 18ms (261 bytes)
1 : 1
Request Properties
Property
Value
Name
Request 1
Description
Properties
Inspector
SoapUI log http log jetty log error log wsrm log memory log
Activate Windows
Go to Settings to activate Windows.
Type here to search
Result
ENG
US
10:28 AM
6/15/2026


SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
REST Project 1
http://localhost:8081
Resource [/RestDemo/api/resou
Resource 1
Request 1
UDDIExample
Request 1
http://localhost:8080/UDDIReqistry
Request 1
Method
Endpoint
Resource
Parameters
GET
http://localhost:8081
/RestDemo/api/resource
Name
Value
Style
Level
Required:
Sets if parameter is required
Type:
Options:
Headers (0)
Attachments (0)
SSL Info
Representations (0)
Schema
JMS (0)
Request Properties
Request Params
Property
Value
Name
Request 1
Description
Properties
SoapUI log
http log
jetty log
error log
wsrm log
memory log
Inspector
Activate Windows
Go to Settings to activate Windows.
Type here to search
ENG
US
10:28 AM
6/15/2026

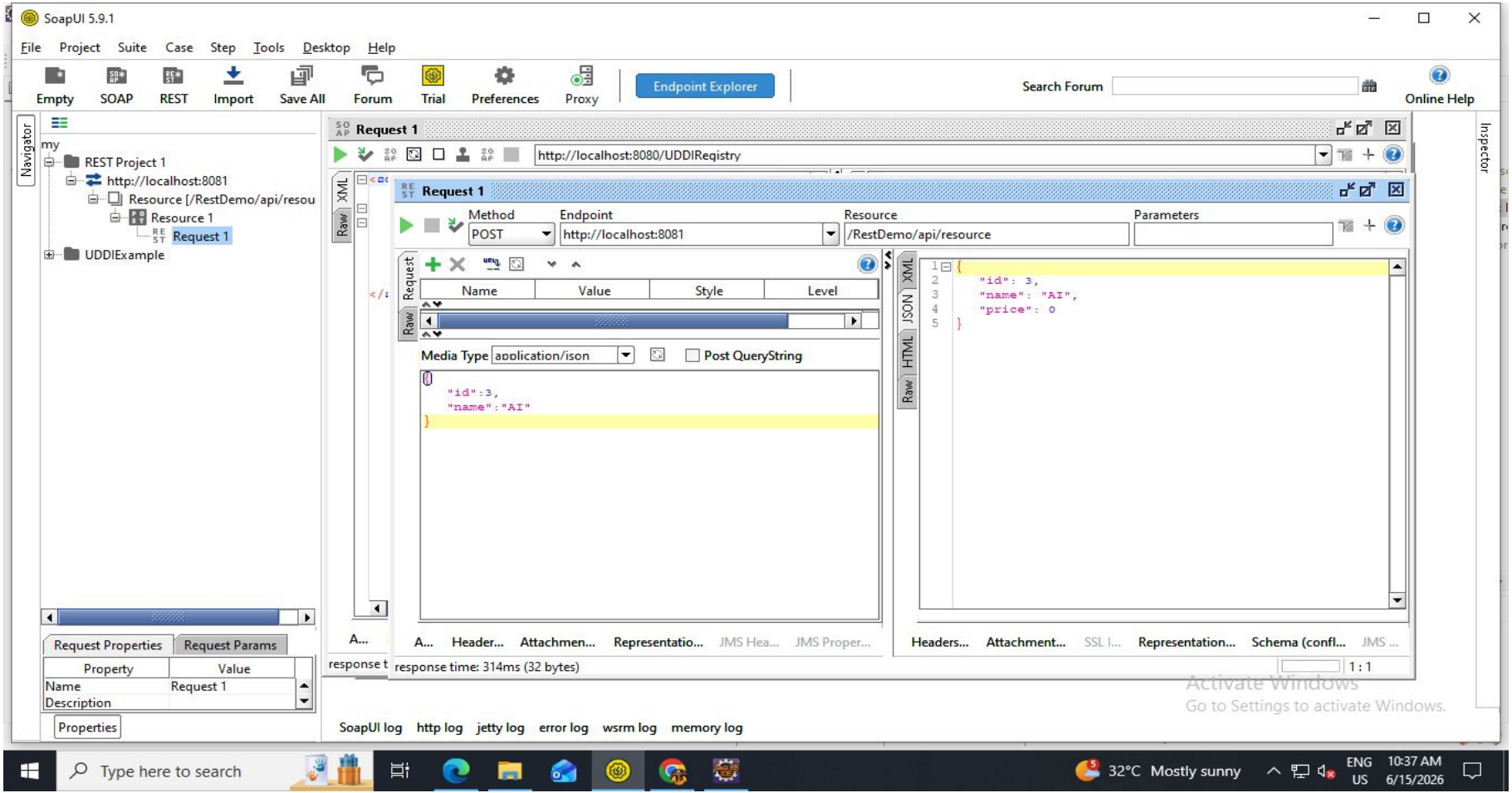

SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
REST Project 1
http://localhost:8081
Resource [/RestDemo/api/resou
Resource 1
Request 1
UDDIExample
Request 1
http://localhost:8080/UDDIRegistry
Request 1
Method
GET
Endpoint
http://localhost:8081
Resource
/RestDemo/api/resource
Parameters
Request
Raw
Name
Value
Style
Level
Required:
Sets if parameter is required
Type:
Options:
XML
JSON
HTML
Raw
[
{
"id": 1,
"name": "Laptop",
"price": 50000
},
{
"id": 2,
"name": "Mobile",
"price": 20000
}
]
A... Header... Attachmen... Representatio... JMS Hea... JMS Proper...
Headers... Attachment... SSL I... Representation... Schema (confl... JMS ...
response time: 12ms (83 bytes)
1 : 1
Request Properties
Request Params
Property
Value
Name
Request 1
Description
Properties
Inspector
SoapUI log http log jetty log error log wsrm log memory log
Activate Windows
Go to Settings to activate Windows.
Type here to search
RELIANCE +1.55%
ENG
US
10:28 AM
6/15/2026

# GET the element by specific ID: 1

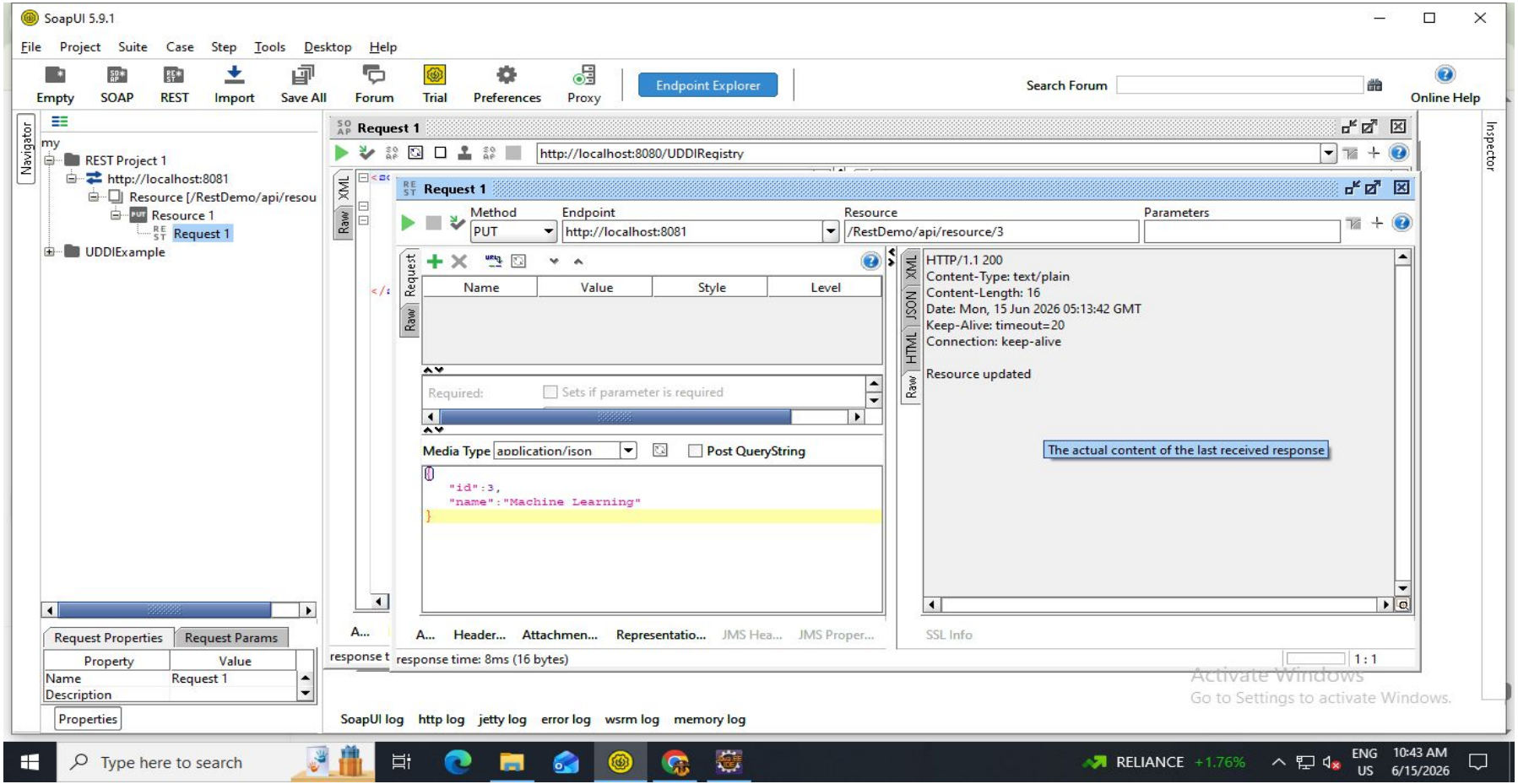



SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
REST Project 1
http://localhost:8081
Resource [/RestDemo/api/resou
Resource 1
Request 1
UDDIExample
Request 1
http://localhost:8080/UDDIRegistry
Request 1
Method
Endpoint
Resource
Parameters
POST
http://localhost:8081
/RestDemo/api/resource/1
Name
Value
Style
Level
Media Type application/json
Post QueryString
{
"id":3,
"name":"AI"
}
{
"id": 1,
"name": "Laptop",
"price": 50000
}
Request Properties
Request Params
Property
Value
Name
Request 1
Description
Properties
response time: 15ms (40 bytes)
SoapUI log
http log
jetty log
error log
wsrm log
memory log
Inspector
Activate Windows
Go to Settings to activate Windows.
Type here to search
This golden retriever...
ENG
US
10:33 AM
6/15/2026

# POST Method Triggered



# Put Method Output: give the resource id as : 3



# DELETE Resource

Suppose resource id=3 exists.

**URL**

http://localhost:8081/RestDemo/api/resource/3

**Method**

DELETE

**SOAP UI Steps**

1. Select **DELETE**
2. URL:
http://localhost:8081/RestDemo/api/resource/3
3. No request body required
4. Click Submit

**Expected Response**

Resource Deleted Successfully

# Resource.java

```
package com.rest;
public class Resource {
   private int id;
   private String name;
   private double price;
   public Resource() {
   }
   public Resource(int id, String name,
double price) {
      this.id = id;
      this.name = name;
      this.price = price;
   }
   public int getId() {
      return id;
   }
   public void setId(int id) {
      this.id = id;
   }
   public String getName() {
      return name;
   }
   public void setName(String name) {
      this.name = name;
   }
   public double getPrice() {
      return price;
   }
   public void setPrice(double price) {
      this.price = price;
   }
}
```

# ResourceRepository.java

```java
package com.rest;
import java.util.ArrayList;
import java.util.List;
public class ResourceRepository {
  private static List<Resource> resources = new
ArrayList<>();
  static {
    resources.add(new Resource(1, "Laptop", 50000));
    resources.add(new Resource(2, "Mobile", 20000));
  }
  public static List<Resource> getAllResources() {
    return resources;
  }
  public static Resource getResourceById(int id) {
    for (Resource r : resources) {
      if (r.getId() == id) {
        return r;
      }
    }
    return null;
  }
  public static void addResource(Resource resource) {
    resources.add(resource);
  }
  public static boolean updateResource(int id,
Resource updatedResource) {
    for (Resource r : resources) {
      if (r.getId() == id) {
        r.setName(updatedResource.getName());
        r.setPrice(updatedResource.getPrice());
        return true;
      }
    }
    return false;
  }
  public static boolean deleteResource(int id) {
    return resources.removeIf(r -> r.getId() == id);
  }
}
```

ResourceService.java

```
package com.rest;
import java.util.List;
import javax.ws.rs.*;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
@Path("/resource")
public class ResourceService {
   // GET ALL
   @GET

@Produces(MediaType.APPLICATION_JSON)
   public List<Resource> getAllResources() {
      return
ResourceRepository.getAllResources();
   }
   // GET BY ID
   @GET
   @Path("/{id}")
   @Produces(MediaType.APPLICATION_JSON)
   public Response getResource(@PathParam("id") int id)
{
      Resource resource =
ResourceRepository.getResourceById(id);
      if (resource != null) {
         return Response.ok(resource).build();
      }
      return
Response.status(Response.Status.NOT_FOUND)
            .entity("Resource not found")
            .build();
   }
   // POST
   @POST
   @Consumes(MediaType.APPLICATION_JSON)
   @Produces(MediaType.APPLICATION_JSON)
   public Response createResource(Resource resource) {
      ResourceRepository.addResource(resource);

      return
Response.status(Response.Status.CREATED)
            .entity(resource)
            .build();
   }
```

```
@PUT
@Path("/{id}")

@Consumes(MediaType.APPLICATION_JSON)
public Response updateResource(
        @PathParam("id") int id,
        Resource resource) {
    boolean updated =

ResourceRepository.updateResource(id,
resource);
    if (updated) {
        return Response.ok("Resource updated")
                .build();
    }
    return
Response.status(Response.Status.NOT_FOUN
D)
            .entity("Resource not found")
            .build();
}
```

```
// DELETE
@DELETE
@Path("/{id}")
public Response deleteResource(
        @PathParam("id") int id) {
    boolean deleted =

ResourceRepository.deleteResource(id);
    if (deleted) {
        return Response.ok("Resource
deleted")
                .build();
    }
    return
Response.status(Response.Status.NOT_FO
UND)
            .entity("Resource not found")
            .build();
}
}
```

# web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="
        http://xmlns.jcp.org/xml/ns/javaee

http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
        version="3.1">
   <servlet>
      <servlet-name>Jersey REST Service</servlet-name>
      <servlet-class>
         org.glassfish.jersey.servlet.ServletContainer
      </servlet-class>
      <init-param>
         <param-name>
            jersey.config.server.provider.packages
         </param-name>
         <param-value>
            com.rest
         </param-value>
      </init-param>
      <load-on-startup>1</load-on-startup>
   </servlet>
   <servlet-mapping>
      <servlet-name>
         Jersey REST Service
      </servlet-name>
      <url-pattern>/api/*</url-pattern>
   </servlet-mapping>
</web-app>
```

# pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="
        http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>CRUDRestService</groupId>
  <artifactId>CRUDRestService</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <dependencies>
    <!-- Jersey Server -->
    <dependency>
      <groupId>org.glassfish.jersey.core</groupId>
      <artifactId>jersey-server</artifactId>
      <version>2.35</version>
    </dependency>
    <!-- Jersey Servlet -->
    <dependency>
      <groupId>org.glassfish.jersey.containers</groupId>
      <artifactId>jersey-container-servlet</artifactId>
      <version>2.35</version>
    </dependency>
    <!-- HK2 Injection -->
    <dependency>
      <groupId>org.glassfish.jersey.inject</groupId>
      <artifactId>jersey-hk2</artifactId>
      <version>2.35</version>
    </dependency>
    <!-- JSON Support -->
    <dependency>
      <groupId>org.glassfish.jersey.media</groupId>
      <artifactId>jersey-media-json-jackson</artifactId>
      <version>2.35</version>
    </dependency>
    <!-- JAX RS -->
    <dependency>
      <groupId>javax.ws.rs</groupId>
      <artifactId>javax.ws.rs-api</artifactId>
      <version>2.1.1</version>
    </dependency>
    <!-- Servlet API -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
  <build>
    <finalName>CRUDRestService</finalName>
  </build>
</project>
```

SOAP UI - CRUDRestService/pom.xml - Eclipse IDE

File Edit Navigate Search Project Run Design Window Help

Project Explorer

- CRUDRestService
  - Deployment Descriptor: CRUDRestService
  - JAX-WS Web Services
  - Java Resources
    - src/main/java
      - com.rest
        - Resource.java
        - ResourceRepository.java
        - ResourceService.java
    - Libraries
  - Deployed Resources
  - build
  - src
    - main
      - java
      - webapp
        - META-INF
        - WEB-INF
          - lib
          - web.xml
  - target
  - pom.xml
- RestDemo
- Servers
- SOAPCalculator

RestDemo/pom... | Student.java | Resource.java | ResourceRepo... | ResourceSer... | web.xml | CRUDRestSer...

```
http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="
         http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>CRUDRestService</groupId>
    <artifactId>CRUDRestService</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <packaging>war</packaging>

    <dependencies>

        <!-- Jersey Server -->
        <dependency>
            <groupId>org.glassfish.jersey.core</groupId>
            <artifactId>jersey-server</artifactId>
            <version>2.35</version>
        </dependency>

        <!-- Jersey Servlet -->
        <dependency>
            <groupId>org.glassfish.jersey.containers</groupId>
            <artifactId>jersey-container-servlet</artifactId>
            <version>2.35</version>
        </dependency>

        <!-- HK2 Injection -->
```

Overview | Dependencies | Dependency Hierarchy | Effective POM | pom.xml

Outline

- Grammars
  - jar:file:/C:/Users/hdadh/eclipse/jee
    - Binding: xsi:schemaLocation wi
    - Cache: false
- project
  - modelVersion
  - groupId
  - artifactId
  - version
  - packaging
  - dependencies
    - dependency
      - groupId
      - artifactId
      - version
    - dependency
    - dependency
    - dependency
    - dependency
    - dependency
  - build

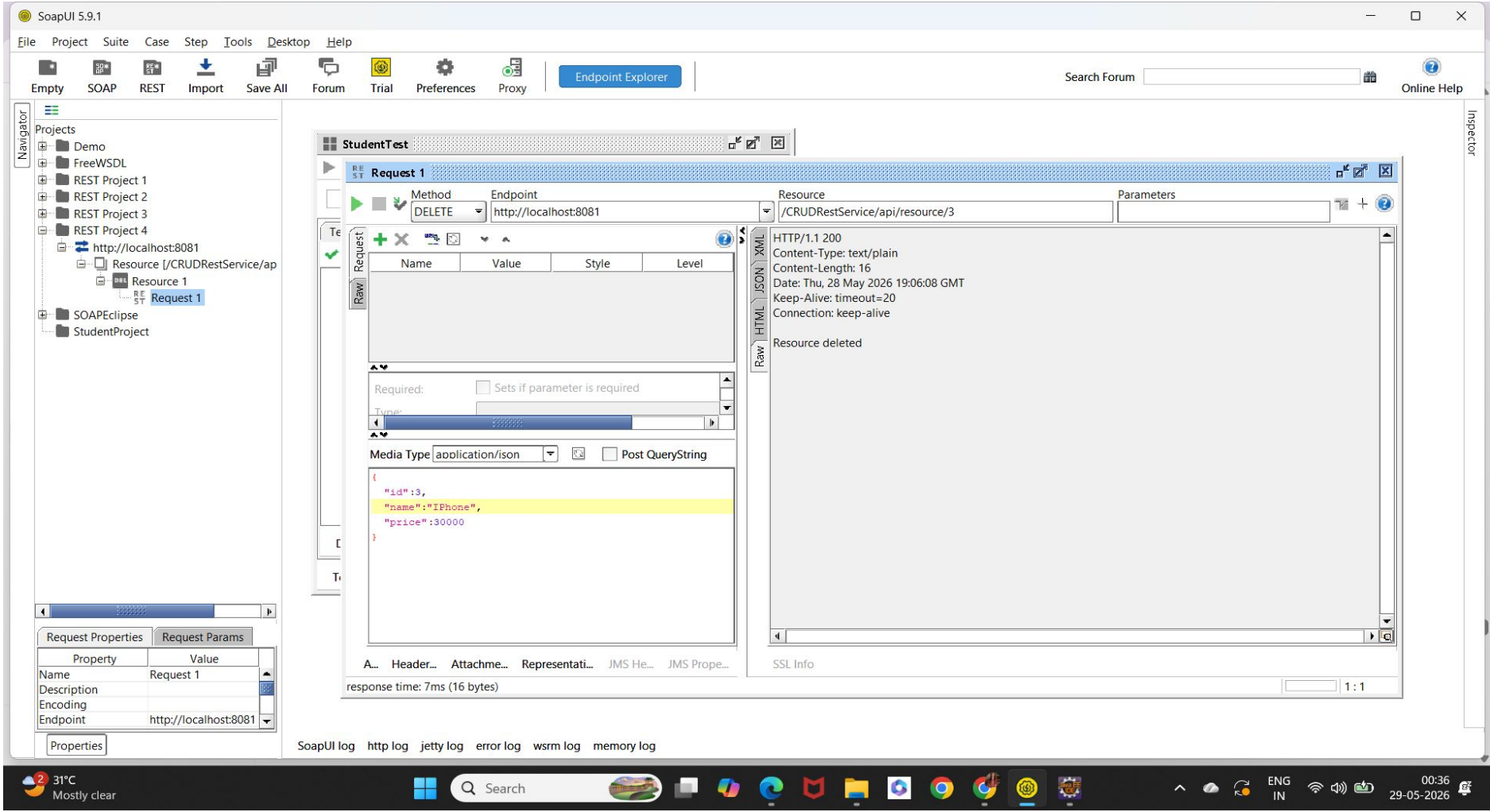
Problems | Servers | Terminal | Data Source Explorer | Properties | Console

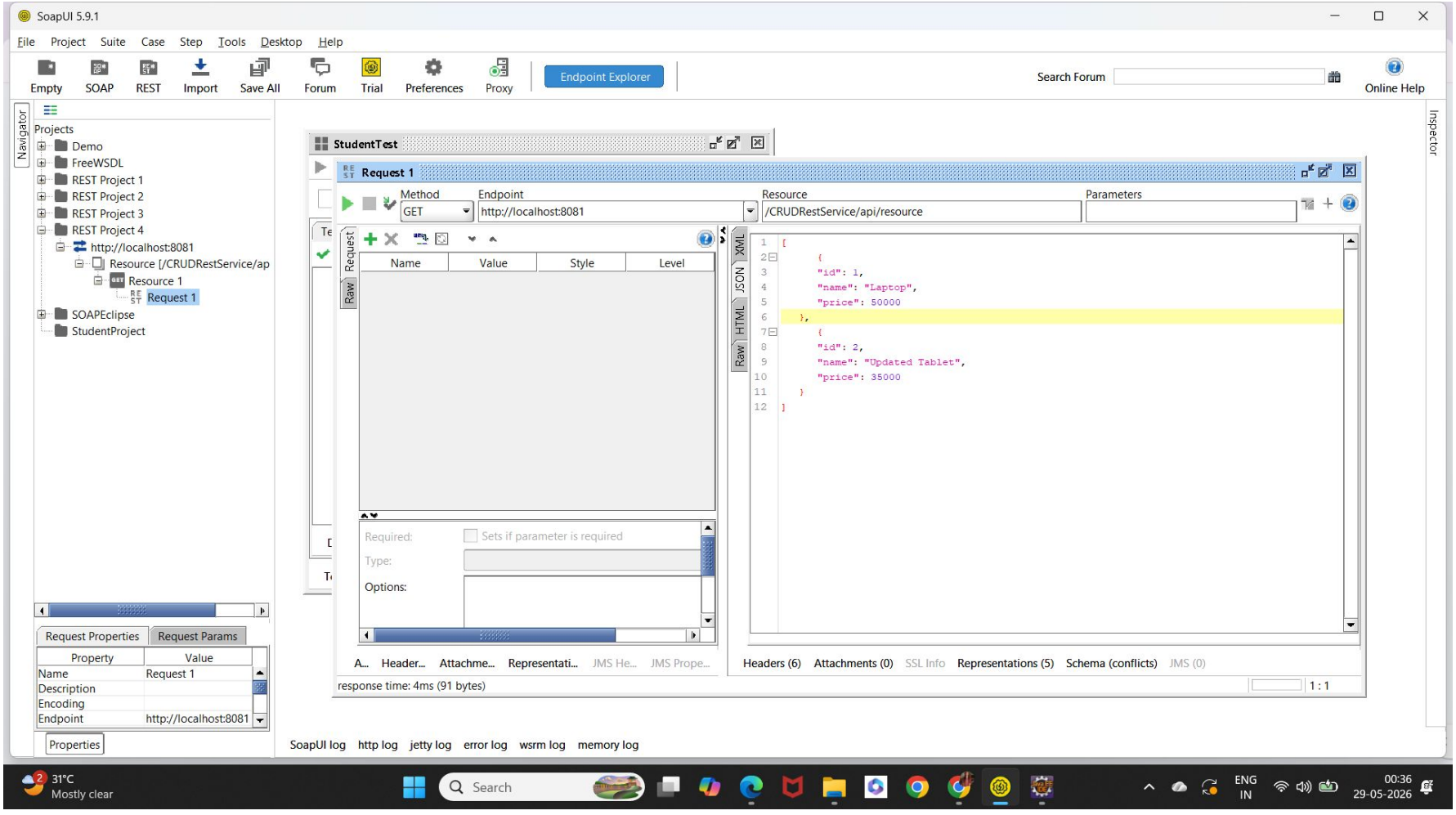
Tomcat v9.0 Server at localhost [Apache Tomcat] C:\Program Files\Java\jre1.8.0_461\bin\javaw.exe (29-May-2026, 12:11:34 am elapsed: 0:35:44) [pid: 8600]

```
May 29, 2026 12:31:25 AM org.apache.jasper.servlet.TldScanner scanJars
INFO: At least one JAR was scanned for TLDs yet contained no TLDs. Enable debug logging for this logger for a complete list of JARs that were scanned but no TLDs were
May 29, 2026 12:31:26 AM org.apache.catalina.core.StandardContext reload
INFO: Reloading Context with name [/CRUDRestService] is completed
```

Writable | Insert | 2 : 63 : 114

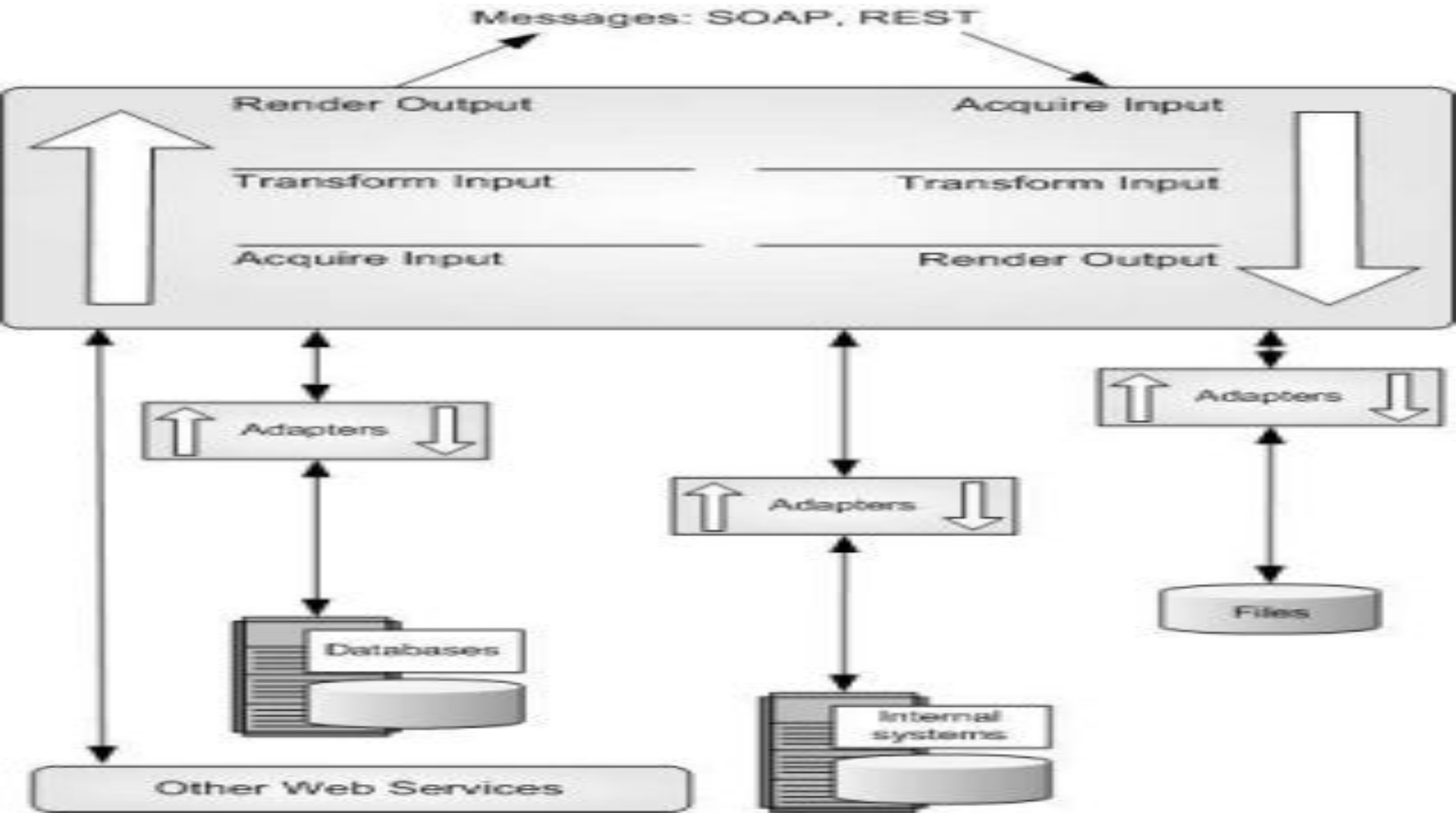
31°C Clear | Search | ENG IN | 00:47 29-05-2026


SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
Projects
Demo
FreeWSDL
REST Project 1
REST Project 2
REST Project 3
REST Project 4
http://localhost:8081
Resource [/CRUDRestService/ap
Resource 1
Request 1
SOAPEclipse
StudentProject
StudentTest
Request 1
Method
DELETE
Endpoint
http://localhost:8081
Resource
/CRUDRestService/api/resource/3
Parameters
Request
Raw
Name
Value
Style
Level
Required:
Sets if parameter is required
Type:
Media Type
application/json
Post QueryString
{
"id":3,
"name":"IPhone",
"price":30000
}
XML
JSON
HTML
Raw
HTTP/1.1 200
Content-Type: text/plain
Content-Length: 16
Date: Thu, 28 May 2026 19:06:08 GMT
Keep-Alive: timeout=20
Connection: keep-alive
Resource deleted
A... Header... Attachme... Representati... JMS He... JMS Prope...
SSL Info
response time: 7ms (16 bytes)
1 : 1
Request Properties
Request Params
Property
Value
Name
Request 1
Description
Encoding
Endpoint
http://localhost:8081
Properties
Inspector
SoapUI log http log jetty log error log wsrm log memory log
31°C
Mostly clear
Search
ENG
IN
00:36
29-05-2026


SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
Projects
Demo
FreeWSDL
REST Project 1
REST Project 2
REST Project 3
REST Project 4
http://localhost:8081
Resource [/CRUDRestService/ap
Resource 1
Request 1
SOAPEclipse
StudentProject
StudentTest
Request 1
Method
GET
Endpoint
http://localhost:8081
Resource
/CRUDRestService/api/resource
Parameters
Request
Raw
Name
Value
Style
Level
Required:
Sets if parameter is required
Type:
Options:
A... Header... Attachme... Representati... JMS He... JMS Prope...
XML
JSON
HTML
Raw
[
{
"id": 1,
"name": "Laptop",
"price": 50000
},
{
"id": 2,
"name": "Updated Tablet",
"price": 35000
}
]
Headers (6) Attachments (0) SSL Info Representations (5) Schema (conflicts) JMS (0)
response time: 4ms (91 bytes)
1 : 1
Inspector
Request Properties
Request Params
Property
Value
Name
Request 1
Description
Encoding
Endpoint
http://localhost:8081
Properties
SoapUI log
http log
jetty log
error log
wsrm log
memory log
31°C
Mostly clear
Search
ENG
IN
00:36
29-05-2026

# Complete SOAP UI Testing Sequence

| Step | Method | URL |
|---|---|---|
| 1 | GET | `/api/resource` |
| 2 | POST | `/api/resource` |
| 3 | GET | `/api/resource/3` |
| 4 | PUT | `/api/resource/3` |
| 5 | GET | `/api/resource/3` |
| 6 | DELETE | `/api/resource/3` |
| 7 | GET | `/api/resource/3` |


Messages: SOAP, REST
Render Output
Acquire Input
Transform Input
Transform Input
Acquire Input
Render Output
Adapters
Adapters
Adapters
Databases
Internal systems
Files
Other Web Services

UDDI is an XML-based standard for describing, publishing, and finding web services.

- UDDI stands for **Universal Description, Discovery, and Integration.**
- UDDI is a specification for a distributed registry of web services.
- UDDI is a platform-independent, open framework.
- UDDI can communicate via SOAP, CORBA, Java RMI Protocol.
- UDDI uses Web Service Definition Language(WSDL) to describe interfaces to web services.
- UDDI is seen with SOAP and WSDL as one of the three foundation standards of web services.
- UDDI is an open industry initiative, enabling businesses to discover each other and define how they interact over the Internet.

UDDI has two sections –

- A registry of all web service's metadata, including a pointer to the WSDL description of a service.
- A set of WSDL port type definitions for manipulating and searching that registry.

UDDI is a registry standard for web services, allowing businesses to publish and discover services over the internet. It provides a centralized directory where developers can search for and obtain information about available web services.

An example of using the UDDI Inquiry API to find a web service with a specific service key:

```
<?xml version="1.0" encoding="UTF-8"?>

<find_service xmlns="urn:uddi-org:api_v2"
serviceKey="uddi:example.com:service_helloWorld">

</find_service>
```

The response may include information about the web service, such as its binding templates, which provide details on how to access the service:

```
<?xml version="1.0" encoding="UTF-8"?>
<serviceList xmlns="urn:uddi-org:api_v2">
 <serviceInfos>
   <serviceInfo serviceKey="uddi:example.com:service_helloWorld">
     <name xml:lang="en">HelloWorldService</name>
     <bindingTemplates>
       <bindingTemplate
bindingKey="uddi:example.com:binding_helloWorld">
         <accessPoint
URLType="http">http://example.com/HelloWorldService.asmx</accessPoin
t>
       </bindingTemplate>
     </bindingTemplates>
   </serviceInfo>
 </serviceInfos>
</serviceList>
```

A business or a company can register three types of information into a UDDI registry. This information is contained in three elements of UDDI.

These three elements are –

- White Pages,
- Yellow Pages, and
- Green Pages.

# White Pages

## White pages contain −

- Basic information about the company and its business.
- Basic contact information including business name, address, contact phone number, etc.
- A Unique identifiers for the company tax IDs. This information allows others to discover your web service based upon your business identification.

Contains basic business information:
Business name
Address
Contact information
Example:
Company: ABC Technologies
Phone: +91-9876543210
Email: abc@company.com

# Yellow Pages

- Yellow pages contain more details about the company. They include descriptions of the kind of electronic capabilities the company can offer to anyone who wants to do business with it.
- Yellow pages uses commonly accepted industrial categorization schemes, industry codes, product codes, business identification codes and the like to make it easier for companies to search through the listings and find exactly what they want.

Category: Banking
Category: Education
Category: Healthcare

# Green Pages

Green pages contains technical information about a web service. A green page allows someone to bind to a Web service after it's been found. It includes –

- The various interfaces
- The URL locations
- Discovery information and similar data required to find and run the Web service.

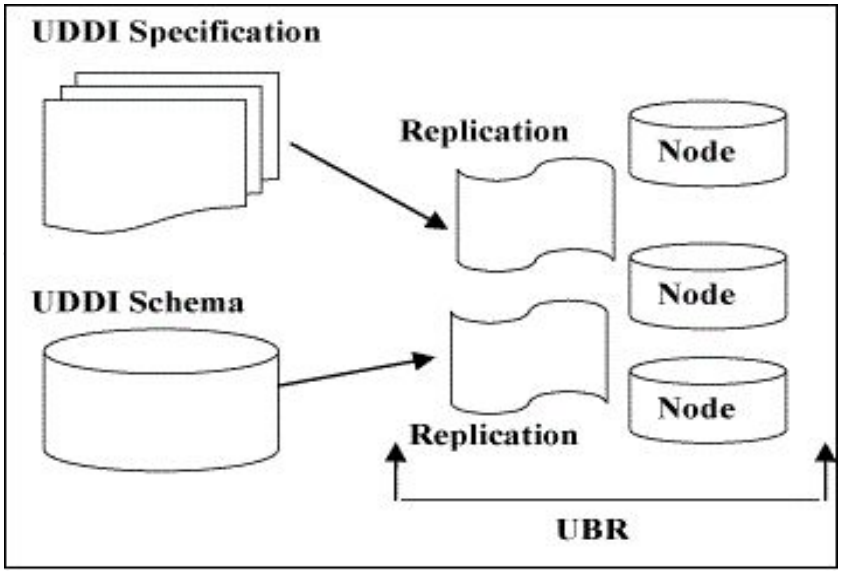
WSDL Location
SOAP Endpoint
Binding Information

**NOTE** – UDDI is not restricted to describing web services based on SOAP. Rather, UDDI can be used to describe any service, from a single webpage or email address all the way up to SOAP, CORBA, and Java RMI services.

- **Definitions** – HelloService
- **Type** – Using built-in data types and they are defined in XMLSchema.
- **Message** –
  - sayHelloRequest – firstName parameter
  - sayHelloresponse – greeting return value
- **Port Type** – sayHello operation that consists of a request and a response service.
- **Binding** – Direction to use the SOAP HTTP transport protocol.
- **Service** – Service available at http://www.examples.com/SayHello/
- **Port** – Associates the binding with the URI http://www.examples.com/SayHello/ where the running service can be accessed.

# UDDI Data Model

UDDI Data Model is an XML Schema for describing businesses and web services




# UDDI API Specification

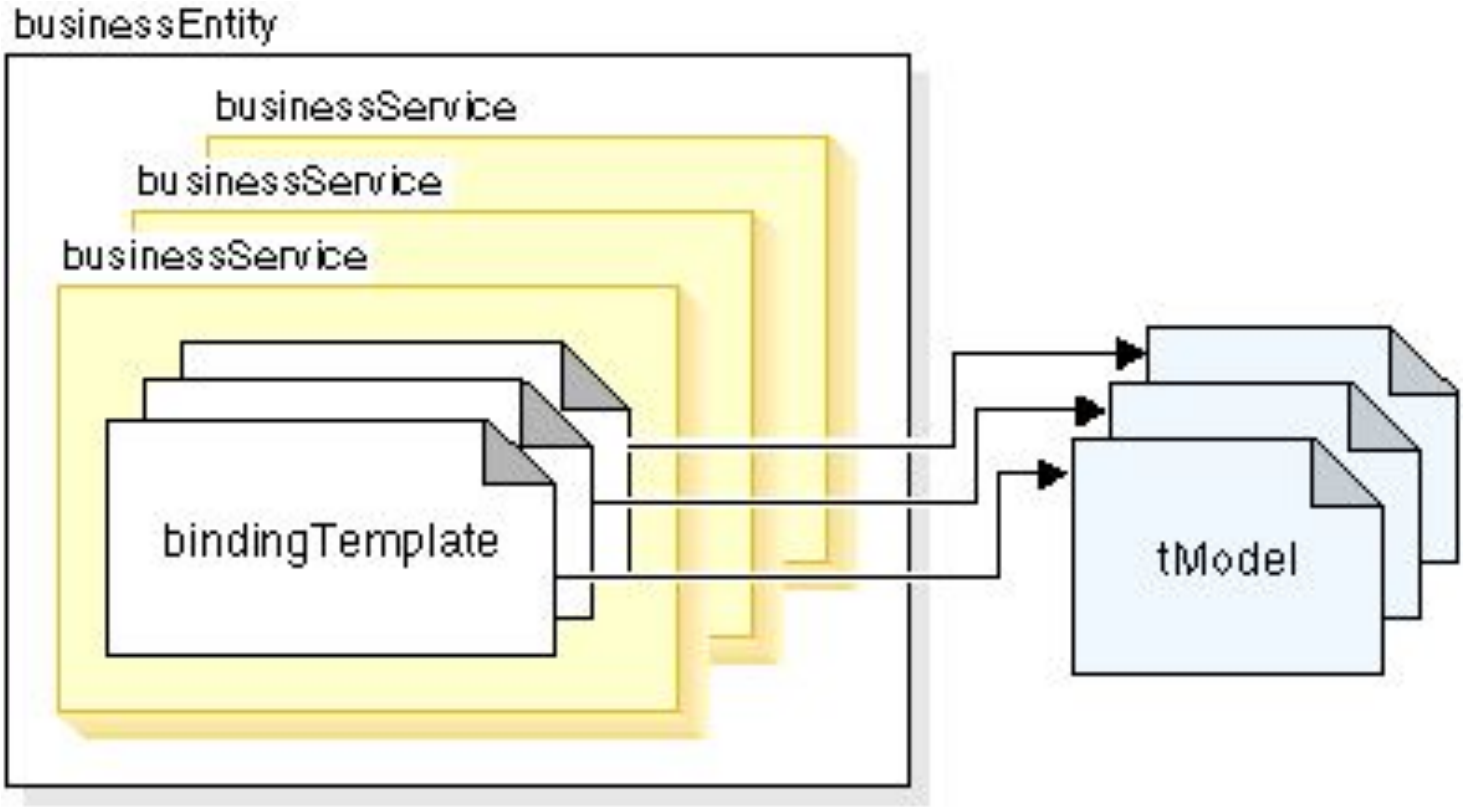
It is a specification of API for searching publishing UDDI data.

# UDDI Cloud Services

These are operator sites that provide implementations of the UDDI specification and synchronize all data on a scheduled basis.

- The UDDI Business Registry (UBR), also known as the Public Cloud, is a conceptually single system built from multiple nodes having their data synchronized through replication.
- The current cloud services provide a logically centralized, but physically distributed, directory. It means the data submitted to one root node will automatically be replicated across all the other root nodes. Currently, data replication occurs every 24 hours.
- UDDI cloud services are currently provided by Microsoft and IBM. Ariba had originally planned to offer an operator as well, but has since backed away from the commitment. Additional operators from other companies, including Hewlett-Packard, are planned for the near future.
- It is also possible to set up private UDDI registries. For example, a large company may set up its own private UDDI registry for registering all internal web services. As these registries are not automatically synchronized with the root UDDI nodes, they are not considered as a part of the UDDI cloud.

https://help.eclipse.org/latest/index.jsp?topic=%2Forg.eclipse.jst.ws.consumption.ui.doc.user%2Fref%2Fruddi.html



# UDDI Registry Data Structure Types

A UDDI registry contains four data structure types that group information about web services:

- **businessEntity** - the top-level data structure that contains information about the business providing the web service. When you find a web service, the business is added to the UDDI browser in the Navigator.

## businessEntity

Represents a business organization.

Example:
<businessEntity>
  <name>ABC Technologies</name>
</businessEntity>
Stores:

- Company name
- Contact information
- Description

**businessService** - contains descriptive information for a family of services, including the name and brief description, and category information.

**bindingTemplate** - contains information about a web service entry point and references to interface specification.

**tModel** - represents the technical specification of the web service. When the Find Web Services Wizard finds a web service, it also displays other web services which are compatible with the same tModel.

# businessService

Represents services offered by a business.

Example:

```
<businessService>
  <name>Online Banking Service</name>
</businessService>
```

Stores:

- Service name
- Service description

# bindingTemplate

Contains technical access details.

Example:

```
<bindingTemplate>
  <accessPoint>
    http://localhost:8080/bankservice
  </accessPoint>
</bindingTemplate>
```

Stores:

- Service URL
- Protocol information

# tModel (Technical Model)

Represents technical specifications.

Example:

```
<tModel>
  <overviewURL>
     bank.wsdl
  </overviewURL>
</tModel>
```

Stores:

- WSDL references
- Interface definitions

# Code Example of UDDI Web services

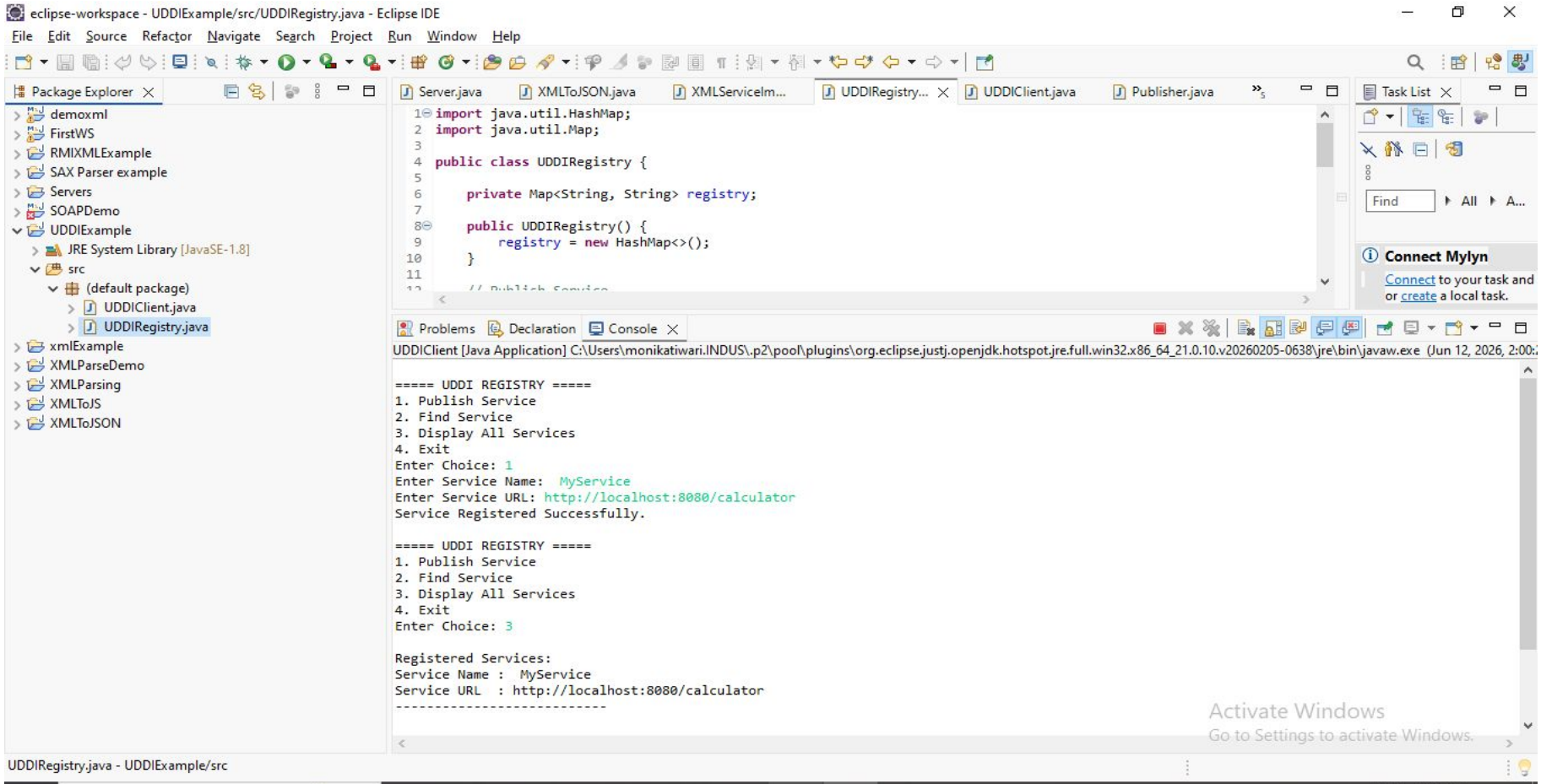
## UDDIClient.java

```
import java.util.Scanner;
public class UDDIClient {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    UDDIRegistry registry = new UDDIRegistry();
    int choice;
    do {
      System.out.println("\n===== UDDI REGISTRY
=====");
      System.out.println("1. Publish Service");
      System.out.println("2. Find Service");
      System.out.println("3. Display All Services");
      System.out.println("4. Exit");
      System.out.print("Enter Choice: ");
      choice = sc.nextInt();
      sc.nextLine();
      switch (choice) {
        case 1:
          System.out.print("Enter Service Name: ");
          String serviceName = sc.nextLine();
          System.out.print("Enter Service URL: ");
          String serviceURL = sc.nextLine();
          registry.publishService(serviceName, serviceURL);
          break;
        case 2:
          System.out.print("Enter Service Name to Search: ");
          String searchName = sc.nextLine();
          String result = registry.findService(searchName);
          if (result != null) {
            System.out.println("Service Found");
            System.out.println("URL: " + result);
          } else {
            System.out.println("Service Not Found");
          }
          break;
        case 3:
          registry.displayServices();
          break;
        case 4:
          System.out.println("Exiting...");
          break;
        default:
          System.out.println("Invalid Choice");
      }
    } while (choice != 4);
    sc.close();
  }
}
```

# UDDIRegistry.java

```
import java.util.HashMap;
import java.util.Map;
public class UDDIRegistry {
   private Map<String, String> registry;
   public UDDIRegistry() {
      registry = new HashMap<>();
   }
   // Publish Service
   public void publishService(String
serviceName, String serviceURL) {
      registry.put(serviceName, serviceURL);
      System.out.println("Service Registered
Successfully.");
   }
   // Find Service
   public String findService(String serviceName) {
      return registry.get(serviceName);
   }
   // Display All Services
   public void displayServices() {
      if (registry.isEmpty()) {
         System.out.println("No services registered.");
         return;
      }
      System.out.println("\nRegistered Services:");
      for (Map.Entry<String, String> entry :
registry.entrySet()) {
         System.out.println("Service Name : " +
entry.getKey());
         System.out.println("Service URL  : " +
entry.getValue());
         System.out.println("-------------------------");
      }
   }
}
```

# Sample Output:



# UDDI Service Testing Using SOAP UI

## Publisher.java

```
package com.uddi;
import javax.xml.ws.Endpoint;
public class Publisher {
   public static void
main(String[] args) {
      Endpoint.publish(

"http://localhost:8080/UDDIRegis
try",
          new
RegistryServiceImpl()
      );
      System.out.println(
         "UDDI Registry Service Started..."
      );
      System.out.println(
         "WSDL Available At:"
      );
      System.out.println(

"http://localhost:8080/UDDIRegistry?wsdl
"
      );
   }
}
```

# RegistryService.java

```
package com.uddi;
import javax.jws.WebMethod;
import javax.jws.WebService;
@WebService
public interface RegistryService {
   @WebMethod
   String publishService(String serviceName, String serviceURL);
   @WebMethod
   String findService(String serviceName);
}
```

# RegistryServiceImpl.java

```
package com.uddi;
import java.util.HashMap;
import java.util.Map;
import javax.jws.WebService;
@WebService(endpointInterface =
"com.uddi.RegistryService")
public class RegistryServiceImpl
implements RegistryService {
   private static Map<String, String>
registry =
         new HashMap<String,
String>();
   @Override
   public String publishService(String
serviceName,
                        String serviceURL) {
      registry.put(serviceName, serviceURL);
      return "Service Registered Successfully";
   }
   @Override
   public String findService(String
serviceName) {
      if (registry.containsKey(serviceName)) {
         return registry.get(serviceName);
      }
      return "Service Not Found";
   }
}
```

eclipse-workspace - SOAPUddi/src/com/uddi/Publisher.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- demoxml
- FirstWS
- RMIXMLExample
- SAX Parser example
- Servers
- SOAPDemo
- SOAPUddi
  - JRE System Library [JavaSE-1.8]
  - src
    - com.uddi
      - Publisher.java
      - RegistryService.java
      - RegistryServiceImpl.java
  - Maven Dependencies
  - bin
  - target
  - pom.xml
- UDDIExample
- xmlExample
- XMLParseDemo
- XMLParsing
- XMLToJS
- XMLToJSON

CourseHandl... RegistryServ... RegistryServ... Publisher.java SOAPUddi/pom...

```
1  package com.uddi;
2
3  import javax.xml.ws.Endpoint;
4
5
6  public class Publisher {
7
8      public static void main(String[] args) {
9
10         Endpoint.publish(
11             "http://localhost:8080/UDDIRegistry",
12             new RegistryServiceImpl()
13         );
14
15         System.out.println(
16             "UDDI Registry Service Started..."
17         );
18
19         System.out.println(
20             "WSDL Available At:"
21         );
22
23         System.out.println(
24             "http://localhost:8080/UDDIRegistry?wsdl"
25         );
26     }
27 }
```

Task List

Find All A...

Connect Mylyn

Connect to your task and or create a local task.

Problems Declaration Console

Publisher (5) [Java Application] C:\Users\monikatiwari.INDUS\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Jun 13, 2026, 4:24

```
UDDI Registry Service Started...
WSDL Available At:
http://localhost:8080/UDDIRegistry?wsdl
```



Writable Smart Insert 18 : 1 : 352

Type here to search

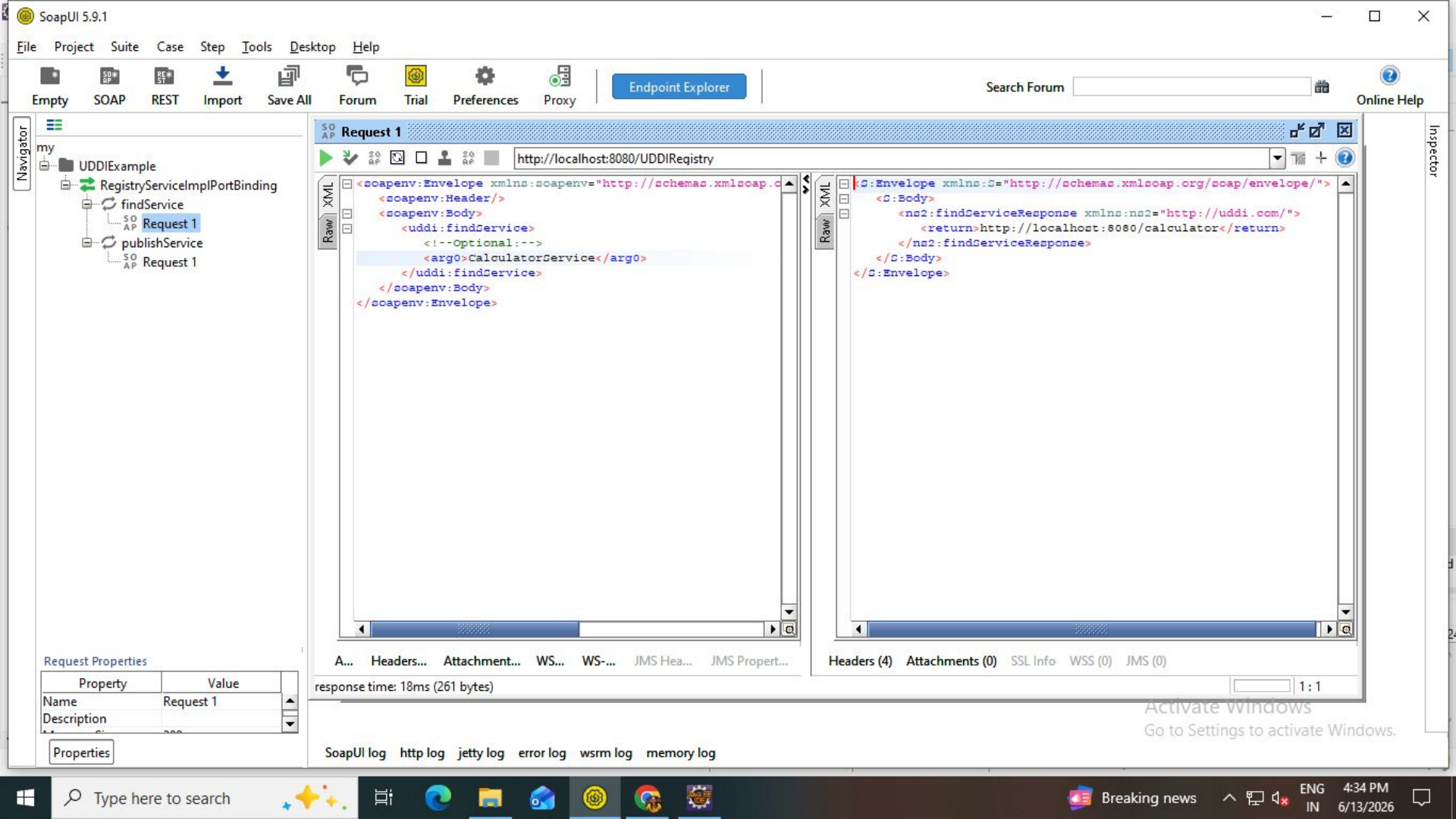
Breaking news

ENG IN 4:33 PM 6/13/2026

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
<!--  Published by JAX-WS RI (https://github.com/eclipse-ee4j/metro-jax-ws). RI's version is JAX-WS RI 2.3.7 git-revision#654ef92.  -->
<!--  Generated by JAX-WS RI (https://github.com/eclipse-ee4j/metro-jax-ws). RI's version is JAX-WS RI 2.3.7 git-revision#654ef92.  -->
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
 xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
 xmlns:tns="http://uddi.com/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://uddi.com/" name="RegistryServiceImplService">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://uddi.com/" schemaLocation="http://localhost:8080/UDDIRegistry?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="publishService">
    <part name="parameters" element="tns:publishService"/>
  </message>
  <message name="publishServiceResponse">
    <part name="parameters" element="tns:publishServiceResponse"/>
  </message>
  <message name="findService">
    <part name="parameters" element="tns:findService"/>
  </message>
  <message name="findServiceResponse">
    <part name="parameters" element="tns:findServiceResponse"/>
  </message>
  <portType name="RegistryService">
    <operation name="publishService">
      <input wsam:Action="http://uddi.com/RegistryService/publishServiceRequest" message="tns:publishService"/>
      <output wsam:Action="http://uddi.com/RegistryService/publishServiceResponse" message="tns:publishServiceResponse"/>
    </operation>
    <operation name="findService">
      <input wsam:Action="http://uddi.com/RegistryService/findServiceRequest" message="tns:findService"/>
      <output wsam:Action="http://uddi.com/RegistryService/findServiceResponse" message="tns:findServiceResponse"/>
    </operation>
  </portType>
  <binding name="RegistryServiceImplPortBinding" type="tns:RegistryService">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
    <operation name="publishService">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal"/>
      </input>
```

```
<dependencies>
    <!-- JAX-WS Runtime -->
    <dependency>
        <groupId>com.sun.xml.ws</groupId>
        <artifactId>jaxws-rt</artifactId>
        <version>2.3.7</version>
    </dependency>
    <!-- JWS Annotations -->
    <dependency>
        <groupId>javax.jws</groupId>
        <artifactId>javax.jws-api</artifactId>
        <version>1.1</version>
    </dependency>
</dependencies>
```


SoapUI 5.9.1
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Trial Preferences Proxy
Endpoint Explorer
Search Forum
Online Help
Navigator
my
UDDIExample
RegistryServiceImplPortBinding
findService
Request 1
publishService
Request 1
Request 1
http://localhost:8080/UDDIRegistry
XML
Raw
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.c
<soapenv:Header/>
<soapenv:Body>
<uddi:findService>
<!--Optional:-->
<arg0>CalculatorService</arg0>
</uddi:findService>
</soapenv:Body>
</soapenv:Envelope>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
<S:Body>
<ns2:findServiceResponse xmlns:ns2="http://uddi.com/">
<return>http://localhost:8080/calculator</return>
</ns2:findServiceResponse>
</S:Body>
</S:Envelope>
A... Headers... Attachment... WS... WS-... JMS Hea... JMS Propert...
Headers (4) Attachments (0) SSL Info WSS (0) JMS (0)
response time: 18ms (261 bytes)
1 : 1
Request Properties
Property Value
Name Request 1
Description
Properties
Inspector
SoapUI log http log jetty log error log wsrm log memory log
Activate Windows
Go to Settings to activate Windows.
Type here to search
Breaking news
ENG IN
4:34 PM 6/13/2026

# SOAP Message Variations

A SOAP message is in one of the following modes, determined formally by the WSDL:

- Document/literal — This is the default message mode in InterSystems IRIS web services (enable the creation and consumption of standards-based SOAP web services) and is the most commonly used mode.
  This message mode uses document-style binding and literal encoding format; bindings and encoding formats are discussed briefly in subsections.
- RPC/encoded — This is the second most common mode.
- RPC/literal — This mode is widely used by IBM.
- Document/encoded — This mode is extremely rare and is not recommended. It is also not in compliance with the WS-I Basic Profile 1.0.

Informally, document/literal messages can have an additional variation: they can be either *wrapped* (the default in InterSystems IRIS) or *unwrapped*. In a wrapped message, the message contains a single part that contains subparts. This is relevant in the case of methods that take multiple arguments. In a wrapped message, the arguments are subparts within this message. In an unwrapped message, the message consists of multiple parts, one per argument. RPC messages can have multiple parts.

# Binding Style

Each web method has a binding style for the inputs and outputs of the web method. A binding style is either document or RPC. The binding style determines how to translate a WSDL binding to a SOAP message. It also controls the format of the body of the SOAP messages.

# How Message Variation Is Determined

For an InterSystems IRIS web service or web client, the details of the service or client class control the message mode used by each web method. These details are as follows:

- The SoapBindingStyle class keyword and the SoapBindingStyle method keyword. The method keyword takes precedence.
- The SoapBodyUse class keyword and the SoapBodyUse method keyword. The method keyword takes precedence.
- The *ARGUMENTSTYLE* class parameter.

The following table summarizes how the message mode is determined for an InterSystems IRIS web method:

When you generate a web service or client class, InterSystems IRIS sets the values for these keywords and parameter as appropriate for the WSDL from which you started.

| Message | SoapBindingStyle | SoapBodyUse | ARGUMENTSTYLE |
|---|---|---|---|
| wrapped document/literal | **`document`** (default) | **`literal`** (default) | **`wrapped`** (default) |
| unwrapped document/literal | **`document`** | **`literal`** | **`message`** |
| rpc/encoded | **`rpc`** | **`encoded`** | *Ignored* |
| rpc/literal | **`rpc`** | **`literal`** | *Ignored* |
| document/encoded | **`document`** | **`encoded`** | *Ignored* |

# AWS (Amazon Web Services)

allows businesses to rent computing power, storage, and databases over the internet

https://docs.aws.amazon.com/hands-on/latest/build-react-app-amplify-graphql/build-react-app-amplify-graphql.html?ref=gsrchandson

## WSDL (Web Services Description Language)

WSDL is an XML-based language used to describe web services and their operations, inputs, and outputs. It provides a way for developers to understand how to interact with a web service without having to access its source code.

**Example: WSDL Document**

A WSDL document includes the following components:

- **Definitions:** Define the data types, messages, and operations.
- **Types:** Define the data types used in the messages.
- **Messages:** Define the structure of the data exchanged.
- **Port Type:** Define the operations and their input/output messages.
- **Binding:** Specify the protocol and encoding style for the operations.
- **Service:** Define the network address and binding information.

SOAP, WSDL, and UDDI are fundamental components of web services, providing a standardized approach for applications to communicate and exchange data. SOAP enables the messaging between applications, WSDL describes the web services and their operations, and UDDI facilitates the discovery and integration of services. By understanding and implementing these technologies, developers can create interoperable and scalable applications that harness the power of the internet.

https://www.tutorialspoint.com/uddi/uddi_elements.htm

https://medium.com/@AlexanderObregon/exploring-web-services-and-xml-soap-wsdl-and-uddi-66353df368d2

Step to Step Guide to use SOAP UI

https://javacodehouse.com/blog/mock-rest-apis-soapui/

Rest Testing

https://www.soapui.org/docs/rest-testing/

# UDDI Life Cycle Management

Life Cycle Management refers to managing web services from creation to retirement.

**Phases**

**1. Publish**

- Register business information
- Publish service descriptions

**2. Discover**

- Search services using UDDI inquiry APIs

**3. Bind**

- Obtain WSDL and endpoint information
- Connect to the service

**4. Invoke**

- Send SOAP requests
- Receive responses

**5. Update**

- Modify service information

**6. Retire/Delete**

- Remove obsolete services

# Dynamic Access Point Management

Dynamic Access Point Management enables web service endpoints to change without affecting clients.

**Need**

- Server migration
- Load balancing
- Cloud deployment
- Disaster recovery

**Working**

1. Service provider updates endpoint information in UDDI.
2. Client queries UDDI.
3. Latest service URL is obtained.
4. Client accesses the current service endpoint.

**Advantages**

- Improved flexibility
- Reduced downtime
- Easier maintenance
- Better scalability

**Example**

Old Endpoint:

http://server1.company.com/service

New Endpoint:

http://server2.company.com/service

After updating UDDI, clients automatically obtain the new endpoint without code changes.

Publish

↓

Discover

↓

Bind

↓

Invoke

↓

Update

↓

Version Management

↓

Retire/Delete